

PBench - Benchmarking Relational Provenance Systems

Pengyuan Li
Illinois Institute of Technology
USA
pli26@hawk.illinoistech.edu

Davood Rafiei
University of Alberta
Canada
drafie@ualberta.ca

Boris Glavic
University of Illinois, Chicago
USA
bglavic@uic.edu

ABSTRACT

Data provenance plays an important role for establishing trust in data, debugging complex computations, understanding query outcomes, and supporting auditing and compliance. Over the last two decades, several relational provenance systems have been proposed, but comparing them remains difficult because they differ not only in runtime, but also in the provenance models they support, the SQL features they handle, how provenance is represented, and how easily captured provenance can be used in downstream tasks. We present PBench, the first benchmark for relational provenance systems that evaluates these dimensions jointly. PBench combines three complementary workload families: *SQL-specific workloads* that stress core relational operators and query structures, *provenance-specific workloads* that vary provenance size and representation characteristics, and *use-case-driven workloads* that evaluate end-to-end tasks such as reproducibility and debugging. Using PBench, we compare state-of-the-art systems including GProM, ProvSQL, SQLProv, and SmokedDuck. Our results reveal clear trade-offs between capture cost, storage overhead, provenance expressiveness, and end-to-end usability. In particular, systems that capture provenance in forms directly consumable by downstream tasks can outperform systems with cheaper or more compact intermediate representations once full task execution is considered. These results provide the first unified experimental picture of the design space of relational provenance systems and highlight open directions for provenance capture, representation, and querying.

PVLDB Reference Format:

Pengyuan Li, Davood Rafiei, and Boris Glavic. PBench - Benchmarking Relational Provenance Systems. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/pli26/PBench_2027.git

1 INTRODUCTION

Relational data provenance has evolved from a largely theoretical concept into a practical tool used in a wide range of data management tasks. Provenance enables users to trace how query results were derived, supporting applications such as debugging incorrect outputs, reproducing analyses, enabling probabilistic reasoning, and maintaining derived data products. Despite decades of research

and the development of numerous provenance models and systems, however, we still lack a principled framework for comparing these systems. This gap makes it difficult to understand the trade-offs between alternative designs and to determine which approaches are best suited for specific use cases.

A key challenge in comparing provenance systems lies in the diversity of their underlying models and implementations. Provenance captures how output data depends on input data and the transformations applied during query execution [15, 25, 32]. In relational systems, this is often studied at fine granularity (lineage) [18–20, 22], where each output tuple is associated with the input tuples that contributed to it. However, systems differ significantly in how they represent and manage this information. Some approaches explicitly materialize input-output relationships [24, 45], while others adopt more compact or expressive representations, such as factorized structures or semiring-based provenance polynomials [6, 55]. In addition, systems rely on diverse architectural techniques—including query rewriting, database extensions, and specialized execution engines—leading to substantial variation in supported SQL features, storage overhead, and runtime performance.

EXAMPLE 1.1. Consider the database shown in Fig. 1 and the following query Q_{emp} that returns the employees with their salary whose departments locates in the region ‘North’.

```
SELECT name, salary
FROM Dept JOIN Emp ON (did = deptid)
WHERE region = 'North';
```

Fig. 2 shows the result of Q_{emp} with lineage and provenance polynomials. The provenance data of results tuples are three tuples from **Emp** (tuples t_1 , t_2 and t_3 shown with purple background) and two tuples from **Dept** (tuples s_1 and s_2 shown with purple background), as the filter condition only outputs departments in ‘North’ and then, query projects the ‘name’ and ‘salary’. The lineage, like t_3 and s_2 for r_3 (shown in dark green), shows the contributed input tuples. The polynomials (shown in blue) represent how the input data is transformed, i.e., for r_1 , tuple t_3 needs to join with s_2 .

Ex. 1.1 illustrates these differences using lineage and provenance polynomials. While lineage directly captures the contributing input tuples for each output, provenance polynomials encode how these tuples are combined during query evaluation. This example highlights two key challenges: (i) different provenance models vary in expressiveness and representation size, and (ii) the choice of representation affects how easily provenance can be used for downstream tasks such as debugging or analysis.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

eId	name	deptId	salary	N[X]	dId	deptName	region	N[X]
1	Alice	101	110,000	t_1	101	Sales	North	s_1
2	Peter	101	110,000	t_2	102	IT	North	s_2
3	Bob	102	80	t_3	103	Marketing	South	s_3
4	John	103	95,000	t_4	104	HR	East	s_4
5	George	104	155,000	t_5	105	Finance	North	s_5

Emp **Dept**

Figure 1: Database relations

	name	salary	Lineage	N[X]	
r_1	Alice	110,000	$\{t_1, s_1\}$	$t_1 \cdot s_1$	x_1
r_2	Peter	110,000	$\{t_2, s_1\}$	$t_2 \cdot s_1$	x_2
r_3	Bob	80	$\{t_3, s_2\}$	$t_3 \cdot s_2$	x_3

Figure 2: Q_{emp} result

These differences have important practical implications. Provenance is used to support a variety of tasks, including tracing erroneous results back to their sources (backward tracing), identifying outputs affected by input errors (forward tracing), explaining complex query behavior, and increasing trust in analytical results. Beyond these user-facing applications, provenance also enables advanced functionalities such as probabilistic query processing [57, 60], hypothetical reasoning (e.g., what-if and how-to queries) [12, 17, 21, 43], reproducibility of computational experiments [16, 58], and efficient maintenance of derived data products under updates [33]. However, as suggested by Ex. 1.1, the effectiveness of provenance for these tasks depends heavily on how it is represented and captured.

Despite its importance, there is currently no standardized benchmark for relational provenance systems. Existing systems are typically evaluated in isolation, often using workloads tailored to highlight their specific strengths. As a result, fundamental systems questions remain unanswered: When does a compact representation hinder usability for downstream tasks? When is the additional cost of a more expressive model justified? How do provenance-capture strategies interact with query structure, provenance size, and end-to-end tasks such as debugging and reproducibility? Prior efforts, such as the provenance challenges introduced in the IPAW workshop series [1, 47], focus on workflows and general computations rather than relational databases, and therefore do not capture database-specific concerns such as complex SQL features (e.g., multi-way joins and aggregation).

To address this gap, we introduce PBench, a benchmark for relational provenance systems. PBench is designed to enable systematic and fair comparison across systems by evaluating them along three complementary axes: (i) SQL features, to measure how provenance overhead varies with query structure; (ii) provenance characteristics, to analyze the impact of representation size and complexity; and (iii) end-to-end tasks, to assess how effectively captured provenance supports practical use cases. This design allows us to expose trade-offs between expressiveness, performance, and usability, rather than focusing solely on provenance capture time.

We design PBench, based on an analysis of provenance models, system architectures, and real-world uses cases. The benchmark includes workloads that stress-test both query processing and provenance management, as well as task-driven evaluations that measure the utility of provenance for applications such as debugging and reproducibility. By combining these perspectives, PBench provides a comprehensive framework for understanding the strengths and limitations of existing provenance systems.

Contributions. We introduce PBench, a benchmark for systematically comparing relational provenance systems, featuring a suite

of workloads that stress both algorithmic and implementation aspects and reflect common provenance use cases. We use PBench to evaluate state-of-the-art systems, including SmokedDuck, ProvSQL, SQLProv, and GProM, on PostgreSQL and DuckDB, revealing key trade-offs: SmokedDuck achieves the most efficient provenance capture due to tight engine integration; GProM performs best on downstream tasks such as debugging owing to its single-pass capture strategy and avoidance of materialized intermediates; and ProvSQL incurs higher overhead due to its use of provenance polynomials. Finally, we derive insights for the design of future provenance systems, showing that design choices should be guided by target use cases, that highly expressive models can introduce unnecessary overhead, and that capture strategies critically impact efficiency—for example, bottom-up approaches may generate large intermediate provenance, whereas top-down strategies can reduce overhead by focusing on relevant data.

The remainder of this paper is organized as follows. In Sec. 3, we discuss motivating use cases for provenance in databases and identify their requirements. We introduce the background on common provenance models in Sec. 2 and given an overview of the systems evaluated in this work in Sec. 4. We present our benchmark PBench in Sec. 5 and use this benchmark to evaluate the performance of systems in Sec. 6. We present conclusions and discuss directions for future work in relational provenance systems in Sec. 9.

2 PROVENANCE MODELS

Since the early work on database provenance, several provenance models have been proposed that provide a formal semantics for explaining query results under different classes of queries [15, 25, 32]. These models differ in at which granularity they track provenance (e.g., tuples or attribute values) and in whether they only track data dependencies (e.g., which input tuple does a query result tuple depend on) or also model how input tuples are combined to produce a result. Furthermore, most models track all alternative derivations of a result while some only track one sufficient one.

In this section, we briefly introduce several provenance models for relational database that are relevant to the systems evaluated in our benchmark. Our goal is not to provide a comprehensive survey, but rather to highlight the key characteristics of these models and discuss the trade-offs they present. A detailed treatment of relational provenance models is beyond the scope of this work; we refer interested reader to existing surveys and introductory articles [15, 25, 32, 35]. The purpose of this section is to provide sufficient background to understand which models are suitable for the use case discussed in Sec. 3 and to highlight the challenges with computing these different provenance types.

	region	Witness	Lineage	Polynomials ($\mathbb{N}[X]$)
r_1	North	$\{t_1, s_1\}$	$\{t_1, t_2, s_1\}$	$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$
r_2	East	$\{t_6, s_4\}$	$\{t_6, s_2\}$	$\delta(t_6 \cdot s_2)$

Figure 3: Result and provenance for Q_{region}

2.1 Witnesses

The concept of a witness for a query result $t \in Q(D)$ was first introduced by Buneman et al. [11] to denote a subset $w \subseteq D$ of the input database D that is *sufficient* for producing t :

$$t \in Q(w)$$

As already observed in [11], witnesses may contain redundant tuples, i.e., tuples that are not needed to produce the result. To avoid such redundancy, we can require witnesses to be *minimal*:

$$\nexists w' \subset w : t \in Q(w')$$

If the goal of provenance is to merely reproduce query results, then using a single witness as provenance is sufficient.¹ However, when there are multiple alternative ways to derive a query result, recording only one minimal witness provides an incomplete picture as we only learn about one alternative derivation.

EXAMPLE 2.1. Consider the following query that computes which regions have departments with employees that earn more than \$100k:

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE salary > 100,000;
```

Evaluating this query over the example database from Fig. 1 returns two results as shown in Fig. 3. Consider the first result tuple r_1 and the witness shown besides the tuple. $\{t_1, s_1\}$ is sufficient for producing the result. However, this is not the only way to derive r_1 : the set $\{t_2, s_1\}$ is a minimal witness as well.

2.2 Lineage

When it is important to identify *all* inputs that contributed to a result as part of some derivation, a single witness is insufficient; instead, a more expressive provenance model is needed. *Lineage*² captures the set of all input tuples that participate in at least one derivation of a result. This concept was first coined in [61] and later formalized based on sufficiency and necessity [22].³ For instance, for the query from Ex. 2.1, the lineage of the first result row is $\{t_1, t_2, s_1\}$ as there are two witnesses (alternative ways to derive this result): $\{t_1, s_1\}$ and $\{t_2, s_1\}$.

¹In fact, in this case we can also use a witness for the complete query result as in [34] where $w \subseteq D$ is a witness if $Q(w) = Q(D)$. We will refer to such witnesses as “holistic” witnesses to distinguish them from witnesses for individual query results. While the distinction to witnesses for individual result tuples seems minor and, in fact, the union of witnesses for all result tuples of a query is a holistic witness, determining minimal witnesses for full query results is unfortunately NP-complete in data complexity, even for some subclasses of conjunctive queries.

²The term lineage is also used in the probabilistic database literature to refer to a different provenance representation, namely Boolean lineage formulas over input tuples. In the K-relational model, this corresponds to semiring $PosBool(X)$. To avoid confusion, we do not use the term lineage for $PosBool(X)$ in this work.

³For some approaches, Lineage is used in a looser sense and may not contain tuples from all possible derivations of a result under some operations, e.g., nested subqueries.

2.3 K-relations and Provenance Polynomials

Green et al. [28] introduced K-relations, a generalization of classical relations where tuples are annotated with elements from a commutative semiring K . A semiring consists of a set K and two binary operations (addition and multiplication) over K that have to fulfill a set of algebraic laws: (i) both operations are commutative and associative, (ii) multiplication distributes over addition, and (iii) there exist neutral elements 0_K and 1_K in K for addition and multiplication respectively. With an appropriate choice of semiring, K-relations can model classical set semantics, bag semantics, and several provenance models including Lineage and MWhy-provenance [11]. Green et al. [28] further demonstrated that the semiring $\mathbb{N}[X]$ of *provenance polynomials*, whose elements are polynomials over variables X representing input tuples, subsumes all provenance models expressible in the K-relational model. Intuitively, using the provenance polynomial semiring $\mathbb{N}[X]$, each input tuple is annotated with a distinct variable. These annotations are propagated during query evaluation such that the annotation of a result tuple t encodes both which input tuples contributed to it and how they were combined to produce t . Here, addition represents alternative derivations (e.g., through projection or union), while multiplication represents joint use of tuples (e.g., through joins).

EXAMPLE 2.2 (PROVENANCE POLYNOMIALS). Consider again query Q_{region} and its result shown in Fig. 3. The provenance polynomial for the first result tuple encodes that there are two alternative ways to derive this tuple, either by joining t_1 with s_1 or by joining t_2 with s_1 . Furthermore, the result was subjected to duplicate elimination (**DISTINCT**, represented as δ in the polynomial).⁴

Provenance polynomials provide a strong guarantee: given the polynomial associated with a query result, we can determine how the result would change if the multiplicities of input tuple were modified. Thus, in a precise sense, provenance polynomials capture the computation that produced the result tuple. Importantly, the polynomials remain valid even when input tuples are deleted or their multiplicity is changed.⁵ Thus, provenance polynomials are critical for provenance use cases involving maintenance of query results and for reverse data management problems [42], where the goal is to determine input modifications that achieve a desired change in the output.

EXAMPLE 2.3 (MAINTAINING RESULTS). Continuing with Ex. 2.2, consider the polynomial of the first tuple:

$$\delta(t_1 \cdot s_1 + t_2 \cdot s_1)$$

Let us assume that we deleted the input tuple t_1 (setting its multiplicity to zero) and increased the multiplicity of the input tuple s_1 to 2 (there are two copies in the input). Will $r_1 = (\text{North})$ still be in the result of the query? To answer this question, we evaluate the polynomial (function δ returns 1 for any non-zero input) by substituting each variable with the multiplicity of the corresponding input tuple:

$$\delta(0 \cdot 2 + 1 \cdot 2) = \delta(0 + 2) = \delta(2) = 1$$

⁴Technically, the δ operator is part of an extension of the semiring framework that models duplicate elimination [5].

⁵Handling the insertion of new input tuples typically requires more expressive provenance models extending the K-relational framework. [30, 31, 38, 63]

Sine the result is non-zero, tuple r_1 remains in the query result after the modification.

2.4 Coarse-grained Over-Approximations

Tracking provenance at the granularity of rows or even attribute values provides detailed information about how query results are derived. However, for some use cases this is overkill, e.g., when we it is enough to have a subset of the input database on which a query produces the same result as on the full database. For such applications, provenance can be captured at a coarser granularity, trading precision for smaller provenance size. For instance, Niu et al. [50] proposes *provenance sketches*, which tracks provenance at the granularity of fragments induced by a horizontal partitioning of a relation. This is also related to summarization of provenance polynomials [2, 3] where multiple variables are replaced with a single coarser-grained

EXAMPLE 2.4. For instance, consider a range-based horizontal partition of table *Dept* on attribute *region*. The coarse-grained provenance of Q_{region} using the partition would contain the two fragments $f_{North} = \{s_1, s_2, s_5\}$ and $f_{East} = \{s_4\}$ as only these two fragments contain provenance. This is an over-approximation of tuple-level lineage as the tuple s_5 does not belong to the lineage of the two result tuples r_1 and r_2 .

3 USE CASES

In this section, we briefly review important use cases for (relational) data provenance, discuss their requirements in terms of provenance models and system functionality, and discuss the properties of typical workloads for each use case which then will inspire our benchmark PBench. Importantly, as discussed in the previous section, provenance models differ in their expressiveness entailing a trade-off between provenance size and cost of capture and the range of use cases that can be supported.

3.1 Debugging Query Results

By tracking which outputs of a query depend on which inputs, database provenance is a useful tool for debugging. In *backward debugging* the user identifies one or more query result tuples of interest that are suspicious or wrong and wants to understand which input data errors are responsible for the observed problem. Provenance supports this use case by limiting the search space as only tuples in the provenance of a result contribute to this result. Backward debugging requires a provenance system that can capture provenance for a subset of the results of a query or, alternatively, to filter the provenance for all query results. For instance, assume that in our running example we consider result tuple r_3 suspicious, because its salary value is unreasonably low. The provenance of this tuple unveils that this unusual result is due to the low salary value of tuple t_3 . In *forward debugging*, some rows in the database are considered problematic and we would like to understand which query results are affected these rows. For instance, if data is imported from multiple sources and one source turned out to be unreliable, then we would like to understand which query results are untrustworthy because they depend on data from this source. Forward debugging necessitates tracing forward from some erroneous inputs to determine impacted query results, i.e., query

results that have problematic inputs in their provenance. This can be done by filtering the provenance of the query to select rows that depend on incorrect source data. Continuing with our running example, assume that the sales department 101 (tuple s_2) is really located in the South (instead of North) region. Once this error is uncovered we would like to determine which query results may be invalidated by this error. A forward provenance query determines that result rows r_1 and r_2 are affected. The existential variant of this type of query is to determine which queries from a workload have some of their results affected.

In terms of provenance models, single witnesses are typically insufficient for debugging as an erroneous result tuple may have multiple alternative derivations of which multiple ones may involve erroneous data. However, witnesses typically contain only tuples from a single such derivation. That is, detecting input errors based on a single witness may not guarantee that all problematic inputs can be identified. Lineage, by including tuples from alternative derivations, is sufficient for debugging. Thus, so are obviously provenance polynomials. Furthermore, provenance polynomials explicitly distinguish between alternative and conjunctive derivations which can be beneficial for understanding which role each input tuple played in generating a query result. Note that we assume that the query itself is correct, and instead we focus on the use of provenance to identify the data that causes incorrect results. Fixing issues with the query requires additional techniques such as query-based explanations for missing answers [9, 10, 14, 23, 59] and query repair [4, 40, 41, 44] which are beyond the scope of this benchmark.

3.2 Reproducing Results

Reproducibility of computational experiments, at the bare minimum requires that rerunning an experiment in a different environment should produce the same result as the original experiment. Provenance enables the creation of “reproducibility packages” [16, 58] which contain sufficient data and instructions of ensure this. For computations that involve a relational database, it would be overkill to include the full database(s) used in the experiment. Instead a reproducibility package should only include subsets of these database(s) that are sufficient for guaranteeing that all statements executed by the experiment return the same result as in the original execution [52, 53]. In terms of provenance model, witnesses can be used for reproducibility as a witness for a query is by definition sufficient for regenerating the result of the query. Thus, lineage and provenance polynomials are sufficient too, but are overkill for this use case.

EXAMPLE 3.1. Assume that query Q_{region} (Ex. 1.1) is part of a computational experiment. A reproducibility package for this experiment only needs to include a witness for this query to ensure reproducibility. For example for Q_{region} , such a witness can be generated by computing the union of the witnesses for all result tuples of Q_{region} (Fig. 3):

$$w = \{t_1, t_6, s_1, s_4\}$$

The reader can confirm that $Q_{region}(w)$ returns the same result as $Q_{region}(w)$ as shown in Fig. 3.

3.3 Probabilistic Databases

Probabilistic databases model data uncertainty through a probability distribution over possible worlds. A commonly used model is tuple-independent databases [60] which associates each tuple with its marginal probability. Provenance plays an essential role in probabilistic databases [57, 60] – provenance polynomial formulas often provide symbolic representation of tuple derivations and can be used to compute the marginal probability of query result tuples under bag semantics. For set semantics, Boolean formulas over variables representing input tuples are sufficient.⁶ Witnesses and Lineage are not sufficient for probabilistic query processing as computing query result probabilities is not possible without knowing how input tuples have been combined.

EXAMPLE 3.2 (PROBABILISTIC DATABASES). *Continuing with our running example, consider the polynomial for result tuple r_1 . By substituting addition with logical or and multiplication with logical and, we get the Lineage formula for this result:*

$$(t_1 \wedge s_1) \vee (t_2 \wedge s_1) = s_1 \wedge (t_1 \vee t_2)$$

Assuming input tuple probabilities $p(t_1) = 0.8$, $p(t_2) = 0.9$, $p(s_1) = 0.5$, the marginal probability of r_1 is the same as the probability of its Lineage formula evaluating to true [60]:

$$p(r_1) = p(s_1) \cdot (1 - (1 - p(t_1)) \cdot (1 - p(t_2))) = 0.5 \cdot (1 - 0.2 \cdot 0.1) = 0.49$$

3.4 View Maintenance, View Update, and Hypothetical Reasoning

Expressive provenance models, i.e., provenance polynomials, enable efficient maintenance and updates of materialized views. In view maintenance, the output of a query should be updated to reflect an update to the database. Similarly, in what-if analysis we want to determine how a hypothetical change to the database impacts the result of a query. For any deletion of a subset of inputs, the provenance polynomial of a query result can be used to evaluate the impact of this deletion on this specific result by replacing the variables for deleted tuples with zero and reevaluating the polynomials. Thus, provenance polynomials provide a solution to maintain views for positive relational algebra queries where updates are restricted to deletions.⁷ In view update (How-to analysis), a desired change to the output of a view should be translated into an update to the database such that running the view’s query over the updated database yields the requested update to the view’s result. If the desired change to the output is a deletion that should be translated into deletion, then this is referred to as *delete propagation* [12, 36]. Using provenance polynomials, finding a solution to a How-to query [42, 43] boils down to constraint-solving: finding an assignment of input variables that determines which tuples to delete in the input such that the polynomials of result tuples evaluate to a desired quantity (does this result tuple exist / select a multiplicity for the result tuple).

Lineage is insufficient for provenance-base view maintenance and update, as it only identifies what row contributed to a result

⁶These formulas correspond to semiring $PosBool(X)$ in the K-relational framework. These formulas are often referred to as Lineage formulas in the probabilistic database literature [60].

⁷With the extensions from [5], aggregate queries are also supported. Furthermore, there are provenance models that also deal with negation and can support insertions [30, 39]

tuple, but not how these input tuples were combined which is necessary for determining how input tuple deletions affect the result of a query.

EXAMPLE 3.3 (WHAT-IF AND HOW-TO). *Continuing with Ex. 2.1. Suppose Alice does not work for the company any more (tuple t_1 is deleted from table Emp). To determine the impact of this update on the result of Q_{region} , we can replace t_1 with 0 in the polynomial for every result tuple. Any result tuple whose polynomial evaluates to zero after this step will no longer be part of the result. Consider the polynomial for r_1 :*

$$\delta(0 \cdot s_1 + t_2 \cdot s_1) = \delta(t_2 \cdot s_1)$$

Thus, r_1 is still part of the result. Note that it is not possible to determine this based on the lineage of r_1 as we do not know how tuples t_1 , t_2 , and s_1 were combined. Now consider a how-to query. We want to delete r_1 from the result of Q_{region} and seek an update to the input database that achieves this. This entails solving the following constraint:

$$t_1 \cdot s_1 + t_2 \cdot s_1 = 0$$

For example, one possible solution is to delete s_1 (setting s_1 to 0). If we want to ensure that r_2 is not deleted, then we need to add another constraint:

$$t_6 \cdot s_2 > 0$$

To minimize the number of deletions (assuming the domain of each variable is $\{0, 1\}$) we need to optimize for the following objective:

$$\max \left(\sum_i t_i + \sum_i s_i \right)$$

3.5 Optimizing Data Management

The provenance of a query is sufficient for answering the query, i.e., it records what data is relevant for the query. Thus, provenance can be used to optimize the performance of operations in a database. For example, using the provenance of a query Q_1 we can speed up a similar query Q_2 by evaluating it of Q_1 ’s provenance [50] or improve the performance of visualization systems to reuse the provenance of a query fueling a visualization to compute the data for a refined visualization when the user interacts with the visualization system [62]. Furthermore, provenance can serve as a more precise alternative of access statistics, e.g., data frequently in the provenance of queries could be moved to faster storage [26]. For these use cases, even witnesses and coarse-grained over-approximations are typically enough. Coarse-grained over-approximations like provenance sketches [50] are attractive because of their compactness.

EXAMPLE 3.4 (OPTIMIZATION). *Assume we have captured coarse-grained provenance as explained in Ex. 2.4 containing the fragments f_{North} and f_{East} for regions “North” and “East”. Consider the following query that differs from Q_{region} for which we captured provenance only in using a stricter WHERE condition. This query can safely be evaluated on the two fragments only as its provenance is guaranteed to be a subset of $\{f_{North}, f_{East}\}$ [50], enabling all data in other regions to be skipped.*

```
SELECT DISTINCT region
FROM Dept d, Emp e
WHERE salary > 115,000;
```

4 SYSTEM OVERVIEW

In this section, we give a brief overview of systems evaluated in this work. We select systems based on several key considerations. Obviously, the system should support relational databases. Furthermore, we only consider systems that exist as reasonably complete implementations. Preferable, the system is still actively developed, but we will not necessarily exclude systems that are not. Furthermore, the goal was to select a representative set of systems regarding the architectural / algorithmic decisions. Fig. 4 compares the selected systems GProM [6], ProvSQL [56], SmokedDuck [45] and SQLProv [48] on their system architecture & how provenance is captured, supported provenance models, supported SQL features, and provenance representation and query capabilities. Fig. 5 shows how these systems capture provenance for example query Q_{emp} .

4.1 GProM

GProM [6] is a database middleware that compiles a SQL dialect enhanced with features for capturing and querying provenance into pure SQL to be executed by a backend database. The system supports multiple database backends including PostgreSQL, SQLite, DuckDB, MonetDB, and Oracle. For positive relational algebra, GProM computes a flat-relational encoding of Provenance Polynomials while for more general classes of queries its provenance model encodes at least Lineage. In addition, GProM supports coarse-grained lineage capture, encoded as provenance sketches. GProM supports provenance for SQL queries as well as transactions [7, 13, 49]. GProM uses a propagation approach for capturing provenance: the user selects which attributes of input tables to treat as provenance and GProM instruments the input query Q constructing a SQL query Q_{rewr} that propagates these annotations such that each result tuple of the instrumented query pairs a result tuple of Q with the selected attributes of tuples from its provenance. GProM exposes provenance capture as a SQL query language feature and, thus, full SQL can be used to query provenance and provenance querying and capture can be combined in a single query. In Fig. 5, attributes `Emp.cid` and `Dep.did` are selected as provenance for input rows. In the result of the instrumented query each tuple is paired with a `cid` and `did` value that encodes which input tuples were joined to derive this result. For example, *(Alice, 110000)* is the result of joining the `Emp` tuple with `cid=1` and department tuple with `did=101`.

4.2 ProvSQL

ProvSQL [56] is a PostgreSQL extension that adds support for computing provenance polynomials encoded as circuits. In these circuits each node is identified through an automatically generated UUID. To enable provenance tracking for a table, the user has to call a UDF `add_provenance` which generates a UUID for each row in the table encoding a variable representing this row in provenance polynomials for query results. Under the hood, this adds a new column `ProvSQL` to the table storing these UUIDs. ProvSQL adds a new background worker process to PostgreSQL that logs circuit gates to mmap files. To capture provenance queries are instrumented by adding calls to UDFs that create new circuit gates and edges by sending messages to the background worker using IPC. In the result of the instrumented query an additional attribute `provsql`

stores the UUID of a gate that is the root of a circuit encoding the provenance polynomial for the result row. ProvSQL provides several helper functions that can be used to reconstruct a string representation of the sub-circuit rooted at a given gate and for evaluating the encoded polynomial in a target semiring. For the later the user has to provide a “provenance mapping” that assigns semiring elements to variables. For instance, for the query shown in Fig. 5, the result tuple for Alice is associated with a gate with UUID u_6 which is a $+$ -gate that is the root of the circuit encoding the polynomial $t_1 \cdot s_1$ for this result.

4.3 Smoked Duck

SmokedDuck [45] extends the execution engine of DuckDB to enable Lineage capture. Lineage tracking in SmokedDuck consists of two phases: (1) during query execution, the instrumented versions of a physical operator in SmokedDuck logs to memory a mapping table that record for each output tuple t of the operator which input tuples (their rowids) of the operator contributed to t . Once lineage has been captured, the user can request the lineage for a particular query result tuple (identified by its rowid) or to get a mapping from output to input rowids for all outputs. For that SmokedDuck joins the mapping tables to trace back from the output to the input tables one operator at a time. Note that it is not strictly necessary to create a mapping table for each operators, e.g., for projection the mapping table is the identify mapping and can be omitted.

4.4 SQLProv

SQLProv [48] captures fine-grained provenance for SQL by instrumenting queries. SQLProv is implemented on top of PostgreSQL, but the approach could be ported relatively easily to other databases. The system to capture both tuple-level provenance as well as attribute-level provenance. Provenance is captured in two phases. In the first phase, the input query is instrumented to create tuple identifiers and to log mapping tables between input and output tuple identifiers for operators. This is done by injecting UDFs into the query that insert tuples into logging tables created by SQLProv which serve a similar purpose as the tuple id mappings in SmokedDuck. In the second phase the input query is instrumented to operate on sets of provenance tuple (or attribute value) identifiers instead of data values. UDF function calls are added to load mappings from the logging tables and computations on values are replaced with union operations on provenance identifier sets. In the result of this instrumented query each row (attribute value) stores a set of input tuple (attribute value) identifiers in its provenance.

4.5 System Comparison

Provenance systems use different strategies to capture and represent the provenance. Among all evaluated provenance systems in this work, GProM and ProvSQL need only one phase to capture the provenance, while SQLProv and SmokedDuck require two phases: in phase I, the systems propagate and materialize the provenance data along with query execution and provenance data can be queried in phase II. But SQLProv and SmokedDuck have different methods to query lineage in the phase II: SmokedDuck uses a backward join approach: back tracing provenance by joining the result tuples with materialized data using row ids, while SQLProv

		ProvSQL	GProM	SmokedDuck	SQLProv
System Architecture	Type Backends	Extension PostgreSQL	Middleware Multiple	Extension DuckDB	Extension PostgreSQL
Add. Storage	DB (Filesystem)	✓(✓)	✗(✗)	✓(✗)	✓(✗)
Capture Approach		log circuits and propagate circuit root ids	bottom-up propagation	I. log mapping + II. top-down backtrace (join)	I. log mappings + II. bottom-up propagation
Provenance Models	Lineage Witnesses Prov. Polynomials Coarse-grained	✓ ✓ ✓ ✗	✓ ✓(SPJ) ✓	✓ ✓(SPJ) ✗	✓ ✗ ✗
Supported SQL Constructs	SPJ (distinct) Aggregation Set operations Nested subqueries Additional features	✓(✓) Agg. final operator only set semantics only ✗	✓(✓) ✓ ✓ CTE, Top-K	✓(✓) ✓ ✓	✓(✓) ✓ ✓ CTE, Windows, Recursive, Top-K
	Updates / Transact.	✓(hyperplane updates)	✓(SI transactions)	✓	✗

Figure 4: Overview of provenance systems: architecture, capture approach, supported provenance models and SQL constructs

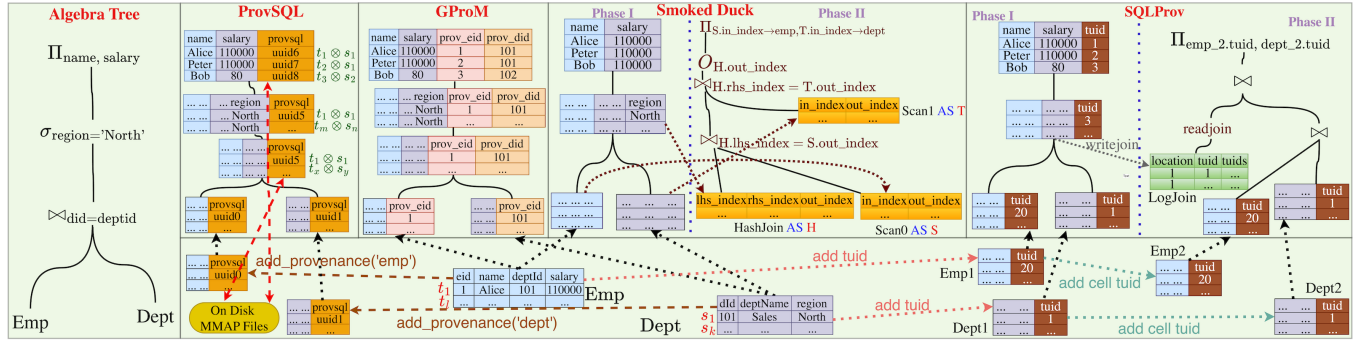


Figure 5: Comparing provenance capture in different systems for example query Q_{emp}

queries the provenance forward: for each query, running the query bottom up to fetch the provenance tuples' tuid using stored data. Systems represent the provenance in different formats: GProM represents the provenance in the form that each provenance tuple associating with its contributed result tuple. For each result tuple, ProvSQL uses one token to indicate the combination of provenance tuples. SQLProv represents a set of tuids (tuples' id) of provenance tuples for each result tuple in phase II reading the materialized the provenance data in phase I. SmokedDuck displays the row ids of provenance tuples. GProM captures the provenance by running rewrite queries and it requires no extra storage during the execution. While extra storage is needed for ProvSQL, SQLProv and SmokedDuck: ProvSQL uses the memory mapped files and it needs many read and write operations dealing with data on disk, and SQLProv and SmokedDuck materialize the data in database tables.

5 PBENCH

We introduce PBench, a benchmark for relational provenance systems. The benchmark consists of multiple task suites, and systems are evaluated based on both task coverage and performance. We first discuss the design considerations, followed by evaluation metrics and data generation process. We then present the benchmark tasks in detail, including their target use cases and the aspect of a provenance system they stress-test. As in prior benchmarks, PBench

defines a scale factor P_{SF} that determines the size of the input database.

5.1 Design Considerations

The tasks in PBench are designed to cover realistic provenance use cases and to evaluate system behavior across diverse provenance characteristics, models, and query constructs. To this end, the benchmark organizes tasks into four categories.

Use-case centric workloads. These tasks evaluate system performance in concrete application scenarios. They simulate realistic workflows that extend beyond provenance capture, emphasizing downstream querying and analysis.

Varying provenance structure / size. Provenance systems differ how they represent provenance, what provenance models they support, and how they capture provenance. This category stress-tests systems under varying provenance characteristics, including overall provenance volume, per-result provenance size, and structural complexity or factorizability of the provenance.

Query complexity. This category evaluates system performance under increasingly complex queries; it assesses how query structure impacts provenance computation and representation.

Fig. 6 shows notation we use for parameters (that are varied in individual tasks) and metrics used in PBench.

Notation	Description
P_{SF}	Scale factor for the benchmark
$P_{\% Q(D) }$	Fraction of query result tuples for which provenance is requested
$P_{\% D }$	Fraction of input tuples which should be traced to a query's result
P_{tot}	Total provenance size for all result tuples of a query
P_{res}	Average size of the provenance of a single result tuple
P_{fact}	Ratio between the size of the sum-of-product of provenance polynomials for a query and the size of polynomials factorized based on the structure of the query
$P_{wit/lin}$	Ratio between the size of a single witness and the size of the full lineage of a query
T_{e2e}	End-to-end runtime (provenance capture + querying + additional task-specific steps)
T_{cap}	Runtime for capturing provenance
T_{query}	Runtime of querying provenance data
T_{task}	Runtime of a task that uses provenance
T_{prep}	Runtime for preparing a database to be able to capture provenance
$S_{capture}$	Additional storage overhead incurred by a system to run a task
S_{db}	Additional storage overhead for preparing a database for provenance capture

Figure 6: Notation for parameters and metrics of the benchmark

Table	#Columns	Number of Rows
R_{fp}^n	7	$n \cdot P_{SF}$
R_{join}^c	4	$5 \cdot P_{SF} \cdot 10^4$
R_{join}^s	4	$25 \cdot P_{SF} \cdot 10^3$
R_{join}^1, R_{join}^2	5	$P_{SF} \cdot 10^4$
$R_{\#grp}^n$	4	$n \cdot P_{SF} \cdot 10^3$
$R_{grpsize}^n$	4	$n \cdot P_{SF} \cdot 10^3$
$R_{fac}^1 \& R_{fac}^2$	5	$P_{SF} \cdot 10^4$
R_{graph}	5	$P_{SF} \cdot 10^5$

Table 1: Synthetic tables used in the benchmark

5.2 Metrics

For each benchmark task, we evaluate systems using two metrics: (1) *execution time* and (2) *storage overhead*. The latter captures the additional storage required—both inside the database and on disk—to maintain the provenance information. Depending on the task, we decompose the total execution time (T_{e2e}) into multiple components: T_{cap} is the time required to capture provenance, T_{query} is the time to query provenance data, and T_{task} is the time to perform downstream tasks using provenance data. Some systems, such as Smokeduck [45, 54] and SQLProv [48], break the process of capturing provenance into two phases: (i) logging sufficient information to (ii) reconstruct provenance based on the logged information. For these systems, we show a breakdown of T_{cap} into $T_{cap-log}$ and $T_{cap-trace}$. Storage overhead ($S_{capture}$) includes all additional data generated for provenance tracking, whether written to disk (e.g., ProvSQL stores provenance circuits as external files) or stored as database tables (e.g., SQLProv logs metadata in relational tables). Furthermore, some systems require augmenting the input database to enable provenance tracking. For these systems, we also report the storage overhead of the augmented base data (S_{db}) and the preprocessing time (T_{prep}) needed to prepare the input database.

5.3 Data Generation

PBench uses the *TPCH* benchmark’s dataset (scaled to P_{SF} selected for PBench) and additional 17 synthetically generated tables whose

size also depends on P_{SF} . Table 1 summarizes the characteristics of all tables used in the benchmark. All attributes in synthetic tables are uniformly distributed. These tables are not intended to model realistic data distribution, but to provide fine-grained control over characteristics such as total provenance size, provenance factorization, and query selectivity. Table R_{fp}^n , for $n \in \{1k, 100k, 2M, 10M\}$, varies the number of distinct attribute values, allowing control over the provenance size per output tuple. Tables R_{join}^c , R_{join}^s , R_{join}^1 and R_{join}^2 contain attributes with at most 1K distinct values and are used in join conditions to control join selectivity and result cardinality. Tables $R_{\#grp}^n$ (and $R_{grpsize}^n$) are designed for aggregation queries, where n controls the number of groups and the number of tuples per group respectively. Tables R_{fac}^1 & R_{fac}^2 are used to control the degree of provenance factorization. Finally, R_{graph} represents an edge relation of a graph with 1K nodes and 10^5 edges, and is used to evaluate recursive queries such as reachability.

5.4 Use-case Specific Tasks

The purpose of these tasks is to test a provenance system on use-case specific workloads. This is relevant as some use cases require further computations on top of captured provenance and how efficiently these operations can be executed depends on how provenance is represented and stored, and what query capabilities for provenance the system provides and how they are implemented.

Some tasks require access to full rows in the provenance and / or query result. For such tasks, systems that return tuple identifiers as provenance have to join back to the input database and / or query result to retrieve the attribute values of these rows. The standard protocol for these tasks is to: (1) capture provenance (2) query the provenance data to retrieve the corresponding rows from the database if needed for the task, and (3) perform additional steps using the retrieved rows to complete the task. Thus, the end-to-end time for a task (T_{e2e}) is the sum of the capture time (T_{cap}), querying time (T_{query}) and additional task-specific computations (T_{task}):

$$T_{e2e} = T_{cap} + T_{query} + T_{task} \quad (\text{Task execution time breakdown})$$

Unless stated otherwise, systems that are able to combine multiple steps are allowed to do so.

5.4.1 Backward Debugging. T1: Backward Debugging simulates a backward debugging scenario. Given a subset of the results of a query that are considered erroneous and should be investigated, the task is to return the actual rows in the provenance of these result rows. This usecase evaluates a provenance system’s capability to determine the provenance for selected result query results. The workflow of backward debugging follows the standard protocol, in which in the last step the system has to filter the provenance to return explicit provenance data for selected result tuples. Systems are allowed to push this selection into provenance capture. We measure the end-to-end performance and the breakdowns described in (Task execution time breakdown). The query we use for this task is *QBDebug* (see Sec. A.1.1) which returns 1K result tuples. We vary the fraction $P_{\%|Q(D)|}$ of query results that are erroneous, i.e., for which provenance is requested (1%, 10%, and 50%).

5.4.2 Forward Debugging. For forward debugging, systems have to determine which query results are affected by problematic input

tuples, i.e., have one of these tuples in their provenance. We vary $P_{\%|D|}$, the fraction of inputs which are considered to be erroneous. PBench contains three tasks for forward debugging that follow the standard protocol. These tasks differ in the task-specific step as explained in the following.

T2: Forward debug - impacted rows requires determining for a given query which result rows are impacted (have erroneous rows in their provenance). Systems are required to return the actual result rows as a user would most likely want to inspect the impacted rows. For this task we use the query templates Q_{FD1} (see Sec. A.1.2), as well as a SPJ query Q_{FD2} (see Sec. A.1.2). For this task, we vary the error rate of tuples, $P_{\%|D|}$, in table $R_{\#grp}^{1k}$ in 1% and 5%. Sec. B.1.1 shows other two interesting forward debugging tasks (1) given a set of queries, determining for each query Q in the set whether it has at least one of the erroneous rows in its provenance, and (2) mapping the erroneous provenance tuples with the affected result tuples.

5.4.3 Reproducibility. In these tasks, the goal is compute a subset of the input database that is sufficient for answering a query as would be required for creating a reproducibility package. Here the provenance system has to return the rows instead of just row identifiers as the reproducibility package should contain the data. For these tasks, systems have to capture provenance for the query of interest and persist the provenance such that the materialized provenance could be included in a reproducibility package ($T_{cap} + T_{query}$). Afterwards, the query should be rerun on the materialized provenance (T_{task}). The last step has to be executed separately as the scenario models creating a reproducibility package ($T_{cap} + T_{query}$) and then using it (T_{task}). The individual reproducibility tasks differ in what provenance model is used as these model represent trade-offs in terms of capture cost and storage size.

T3: Reprod. - lineage requires the reproducibility to be done using lineage. We use tables R_{fp}^{10M} and R_{join}^c and the query template, Q_{Lin} (see Sec. A.1.3). **T4: Reprod. - witness** reproduces the query result using a single witness. We use a query template, Q_{Wit} (see Sec. A.1.3), which differs from T3 only in that we vary $n \in \{1k, 100k, 2M\}$ to control $P_{wit/lin} \in \{0.01\%, 1\%, 20\%\}$ the ratio of the size of a witness versus the size of the full lineage. Sec. B.1.2 presents reproducibility using coarse-grained provenance.

5.4.4 Provenance Polynomials. Use cases such as probabilistic query processing and hypothetical reasoning (what-if and how-to queries) need access to the detailed information of how query results have been combined encoded in provenance polynomials. While provenance provides the foundation for these use cases, they require additional functionality that goes beyond capturing and querying provenance that is only supported by some systems, e.g., FADE [46] based on SmokedDuck supports what-if queries and ProvSQL supports computing probabilities. In task **T5: Provenance Polynomials** the provenance system has to compute provenance polynomials for several queries: $Q_{join[1]}$, $Q_{join[2]}$ and $Q_{join[3]}$ that are SPJ queries, varying the number of joins, and $Q_{joinagg}$, an SPJA query (see Sec. A.1.4). We measure T_{cap} , the runtime for capture, and $S_{capture}$, the storage overhead for capture. In this task we measure T_{cap} , including $T_{cap-log}$ and $T_{cap-trace}$ for ProvSQL and SmokedDuck.

5.5 Provenance Structure & Size

The purpose of the tasks in this group is to evaluate how structural properties and size of the provenance of a query impact system performance. For the tasks in this part, we focus excursively on the runtime of capturing provenance (T_{cap}). Systems are only required to recording sufficient information during query execution to be able to reconstruct provenance later on. Thus, for systems such as SQLProv and SmokedDuck that split capture into two phases (recording sufficient information in the first phase during query execution and reconstruct provenance in the second phase), we only record $T_{cap-log}$, the cost of the first phase.

5.5.1 Provenance Size per Result Tuple. In **T6: Prov. per Result** we fix the total provenance size P_{tot} and vary the provenance size for each query result tuple P_{res} . For that we use two queries, Q_{FPAgg} and Q_{FPDist} (see Sec. A.2.1), over table R_{fp}^{10M} . To vary P_{res} from 10, 10k to 1M, we fix the size of the input tables and control the number of groups (or distinct values of the result attributes in the case of **DISTINCT**): $1M \cdot P_{SF}$, $1K \cdot P_{SF}$, and $10 \cdot P_{SF}$ by selecting the group-by / distinct attribute A_{grp} . Systems differ in how they internally represent the mapping between an (intermediate) result tuple and its provenance and this determines whether the systems will be impacted by a change in P_{res} even if we keep P_{tot} constant. For example, SQLProv does encode the set of tuple identifiers in the provenance of a query result as a single ARRAY value. The size of such an array depends on P_{res} . At the other end of the spectrum ProvSQL produces circuits of almost the same size independent of P_{res} when P_{tot} is fixed.

5.5.2 Varying Total Provenance Size. In **T7: Ttl. Prov. Size**, we vary the total amount of provenance (number of rows) P_{tot} across all result tuples of a query. The purpose of this task is to test how the system deals with large amounts of total provenance information. We vary P_{tot} from $100K \cdot P_{SF}$ to $10M \cdot P_{SF}$ using two queries. For Q_{VPGN} we fix P_{res} to 1K and vary the number of query results ($\{100, 1k, 10k\} \cdot P_{SF}$) by controlling n . For Q_{VPGS} , we fix the number of result tuples to 1K and vary P_{res} ($\{100, 1K, 10K\} \cdot P_{SF}$) by setting $n = \{100, 1K, 10K\}$.

5.5.3 Varying Intermediate Provenance Size. In **T8: Interm. Prov. Size**, we control $P_{spurious}$ the ratio between the size of the provenance of intermediate query results and the total size of the provenance of the final query result (P_{tot}), e.g., if $P_{spurious} = 3$ then the size of the provenance for intermediate query results is three times larger than P_{tot} . For this task, we consider two query templates. Q_{Sel} (see Sec. A.2.3) is an aggregation query with **HAVING** where we vary selectivity ($\{10\%, 50\%, 100\%\}$) by varying the threshold s , resulting in $P_{spurious} = \{\sim 10, \sim 2, \sim 1\}$. In the second query template $Q_{Join\#n}$ (see Sec. A.2.3) we vary the number of joins. $Q_{Join\#3}$ is shown below. $Q_{Join\#1}$ and $Q_{Join\#2}$ are derived from this query by removing the last two (the last) relation and corresponding join conditions, resulting in $P_{spurious} \approx 10/\sim 50/\sim 100$ and $P_{tot} = 10M \cdot P_{SF}$.

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grp1k, Rjoins, [Rjoinj, Rjoinjj]
WHERE ga = p10 [AND ga = c5 AND ga = cc2]
GROUP BY ga
```

5.5.4 Provenance Polynomial Factorization. **T9: Factorization** tests a systems capability of to exploit the potential for factorization of provenance polynomials and, thus, cannot be completed by systems that do not support provenance polynomials. For that we vary P_{fact} , the ratio between the total size of the provenance polynomials for all query results and the size of these polynomials factorized based on the structure of the query. This is measured as the fraction of the size $\#p$ (number of variables) of factorized provenance polynomials⁸ divided by the size $\#SOP(p)$ (number of variables) of the provenance polynomial encoded as a flat sum of products. For example, consider the factorized polynomial $p_1 = (x_1 + x_2 + x_3) \cdot (x_4 + x_5)$ and its sum of product form $SOP(p_1) = x_1x_4 + x_1x_5 + x_2x_4 + x_2x_5 + x_3x_4 + x_3x_5$:

$$\#p_1 = 5 \quad \#SOP(p_1) = 12 \quad P_{fact} = \frac{5}{12}$$

To control the size of the factorized provenance, we consider two queries: (i) Q_{JD} (see Sec. A.2.4) is a duplicate elimination on top of a join while Q_{DJ} (see Sec. A.2.4) additionally eliminates duplicates before the join. The first yields a flat sum-of-products $\sum r_i \times s_i$ while the second one (for systems that support it) yields a factorized polynomial $(\sum_i r_i) \cdot (\sum_j s_j)$ where each r_i (s_i) is a variable for a tuple in R (S). By selecting which attribute A is returned and attribute B we join on we control P_{fact} , ranging from $\frac{1}{10K P_{SF}}$ to $\frac{1}{10P_{SF}}$.

In addition, we also consider two variants of an aggregation query: Q_{JA} (see Sec. A.2.4) computes an aggregate over a join result resulting in a SOP polynomial $\delta(\sum r_i \times s_i)$ while Q_{AJ} pushes the aggregation through the join, resulting in factorized provenance of the form $\delta((\sum_i r_i) \cdot s_i)$.

5.6 Query Constructs & Structure

Tasks in this part of the part of the benchmark evaluate the performance of provenance capture (T_{cap}) and storage overhead ($S_{capture}$) varying the structure of queries and what query constructs (e.g., joins, aggregation, ...) are used. For these tasks we only require systems to capture sufficient information to be able to later restore provenance.⁹ This means that systems that split the capture process into two phases like SQLProv and SmokedDuck are allowed to only report the runtime of the first phase ($T_{cap-log}$).

5.6.1 TPC-H. The *TPCH* benchmark is an well-known benchmark for decision support queries. We include these queries as they contain challenging language features (e.g., nested subqueries) and as a standard benchmark are well understood. **T10: TPC-H** consists of capturing provenance for all 22 queries of the benchmark. As mentioned above the scale factor for *TPCH* is set to P_{SF} (the scale factor for PBench).

5.6.2 Top-K. In **T11: Top-k** the system has to capture provenance for a query that returns the k largest results of an aggregation with group by. This type of query produces large provenance for intermediate results (every input tuple contributes to one group in

⁸Of course, a polynomial permits many possible factorizations that may vary widely in terms of size. We use the factorization that results from the structure of the input query as a reference point as discussed further when we introduce the corresponding benchmark task. All major provenance systems that can exploit factorization, generate a factorization that is based on the query structure.

⁹Just recording what queries are executed is not considered sufficient. That is, the information captured here should record actual provenance relationships at the data level, but it is acceptable if an additional post-processing step is needed to merge the recorded information to produce the full provenance of a query.

the result of the aggregation), but the final provenance of the query is relatively small (only rows contributing to groups that are in the top-k). Systems that split provenance capture into two phases and backtrace provenance top-down in the second phase (SmokedDuck) are expected to perform well for this style of query as the second phase starts from the small final result. In this task we use query template Q_{TopK} (see Sec. A.3.1) which accesses a single table R_{fp}^{10M} , and set k such that 0.01%, 0.1% or 1% of the total $1M \cdot P_{SF}$ groups are returned ($P_{spurious} \in \{0.01\%, 0.1\%, 1\%\}$).

5.6.3 Multi-Level Aggregation. **T12: Multi-level Aggr.** is a multi-level aggregation query. Such queries challenge provenance systems like GProM that compute provenance bottom-up through propagation as it significantly increases the size of intermediate results. We use query Q_{MLAgg} (see Sec. A.3.2) shown below over table R_{fp}^{10M} . The first-level aggregation returns $1K \cdot P_{SF}$ groups which are then further aggregated into a single row by the outer aggregation.

Sec. B.2 presents additional design for varying structure of queries and query constructs, such as argmin and window functions.

6 EXPERIMENTAL EVALUATION

We now use PBench to compare the following provenance systems: SmokedDuck [45], ProvSQL [56], SQLProv [48], and GProM [6]. Systems that run on PostgreSQL are ProvSQL, SQLProv, and GProM. DuckDB is supported by GProM and SmokedDuck. For SQLProv and SmokedDuck that split capture into two phases, the blue percentage numbers on top of bars is the percentage of the total capture time spend in the first phase. For tasks that require rows in the provenance to be returned, systems that represent provenance as row ids, i.e., all systems except GProM, have to fetch the rows with these ids from the database. All experiments were run on a machine with a Apple M2 Max CPU (12 cores) and 64GB RAM running macOS (Darwin Kernel 23.6.0), PostgreSQL 16.3 and DuckDB 1.0.0¹⁰

6.1 Preparing the Database For Capture

Before comparing all systems on the benchmark tasks, we first evaluate the storage and runtime overhead that systems incur for preparing the input database for provenance capture. ProvSQL and SQLProv are the only two systems evaluated in this work for which this step is necessary. For ProvSQL we have to add a provenance mapping to each input table that associates with each input row a variable (that will represent the tuple in provenance polynomials) encoded as a UUID. For SQLProv we have to create a copy of each table whose values are identifiers that are propagate as provenance in the second phase of provenance capture for a query.

The original data base size is ~ 3.2 GB for $P_{SF} = 1$. ProvSQL needs ~ 122 seconds to add provenance token columns to all relations in the database, consuming ~ 2.2 GB of additional disk space. SQLProv takes ~ 104.75 seconds to build the provenance version of all tables and adjust the base version of each table. The total storage size of tables used for capture provenance for SQLProv is ~ 10 GB (database size is ~ 13.3 GB). For *TPCH SF1* datasets, SQLProv runs ~ 39 seconds and adds ~ 5 GB (before: 2.27 GB and after 7.33 GB) in storage to construct provenance related tables.

¹⁰As this is the duckdb version SmokedDuck is based on.

Finally, ProvSQL require ~ 37 seconds to add the provenance token column to all tables, requiring ~ 0.4 GB (before: 2.11 GB and after: 2.50) storage. In summary, the two systems that have to modify the input database to enable provenance tracking introduce significant storage overhead.

6.2 Use-case Specific Tasks

T1: Backward Debugging. Fig. 7a shows the end-to-end runtime T_{e2e} for backward debugging, i.e., retrieving rows in the provenance for a fraction $P_{\%|Q(D)|}$ of query result tuples. Systems that can limit provenance tracking to a subset of query results and that have the flexibility to represent provenance as actual rows instead of rowids have an advantage here as mentioned above. In fact, GProM is the only system with these capabilities¹¹ and its runtime scales in $P_{\%|Q(D)|}$ while the runtime of the remaining systems is roughly constant as the provenance for all results has to be captured first before retrieving relevant rows from the input database based on rowids. Fig. 7b shows the breakdown of runtimes for the individual steps for systems that have to solve this task in multiple steps. As the runtime is dominated by capturing provenance (T_{cap}), these systems do not see any meaningful performance improvements when only the provenance of some results should be tracked. While SmokedDuck is the most efficient system when only considering the first capture phase ($T_{cap-log}$), the overhead of backtracking in the second phase outweighs this advantage.

T15: Forward Debugging and T2: Forward Debugging. ?? shows the end-to-end runtime for determining whether at least one result of a query depends on input data that is considered erroneous. The runtime of systems is again dominated by provenance capture as for backward debugging so we omit the breakdown here (shown in ?? in Sec. C). Fig. 7c shows the runtime for determining which result rows of a query have erroneous rows in their provenance. We vary $P_{\%|D|}$, the fraction of input rows that should be tracked (SmokedDuck is omitted for this experiment, see Sec. D.1.4).

T3: Reproducibility. and **T4: Reproducibility.** Fig. 7e and 7f show the runtime of query Q_{Lin} for using lineage versus a single witness. These figures shows (1) the end-to-end performance of all systems, and (2) performance breakdowns of systems SQLProv, ProvSQL and SmokedDuck. ProvSQL needs to read from and write to memory mapped files, and thus this leads worst performance among all systems. The minimal witness model requires least cost of time and storage to complete the reproducibility tasks, because the amount of total provenance data becomes much less compared to lineage model which has the worst performance. For SmokedDuck and SQLProv, the percentage of phase I over total capture time increases when size of lineage decreases, because (1) phase II is more sensitive to the size of materialized data, and less lineage means less computation in phase II, and (2) phase I is a must-have phase even if a query produce no results. In terms of storage usage, lineage model is obviously largest. Note that for minimal witness, SmokedDuck consumes storage but data size is less than one block, and thus, actual value cannot be queried. Minimal witness model is

an optimized for reproducing the result (reproduce cost) in terms of both execution and storage cost.

??

T5: Provenance Polynomials. Fig. 7g shows the experimental evaluation for systems support this model. For $Q_{join[1]}$, $Q_{join[2]}$ and $Q_{join[3]}$ (first three subfigures), the cost increase due to (1) the increasing of result size, and (2) the increasing of polynomials in each result tuple. The last plot shows the systems that support the query type of $Q_{joinagg}$. For SmokedDuck, the first three evaluations shows that the phase I is relative stable. Compared the first and last, the total costs are relatively the same but phase I has more impact for a aggregation query, since for aggregation there are less result tuples needed to be joined which leads to less cost in phase II (higher percentage for phase I)

Insights: All the experiments of provenance models and use cases demonstrate that among all evaluated systems, GProM performs the best for most use case tasks, because when capturing the provenance, GProM not just propagate the result for each operator but also associates provenance data with result tuples. Thus, further utilization of provenance data can be done by adding operations on top of capture result, such as for debugging, and we can query the needed data directly from results. ProvSQL captures the provenance in one phase, while it is expensive to further use the provenance data, because there is one extra step, querying provenance data from capture result, before the use of provenance. What is more, ProvSQL itself is expensive to capture provenance, because during this process, it needs to read from and write to files on dist. SQLProv and SmokedDuck complete provenance capturing in two phases: materializing operators' provenance (IDs of provenance data) in phase I and inferring provenance for result in phase II. For SmokedDuck, the phase I is much cheaper while phase II is expensive since the system uses join operations to infer the provenance. Like ProvSQL, utilization of provenance captured from SQLProv and SmokedDuck cannot be directly applied for the use cases because these systems capture provenance using tuple IDs, extra querying step needed.

In general, to apply provenance data to implement use cases, like debugging, our experiment demonstrate that GProM is relative easy to use and is cheap to finish the task.

6.3 Provenance Structures & Size

In this section, we evaluate systems focusing on provenance characteristics. We vary parameters like size of per result provenance, provenance factorization of representation to examine how systems behaves when capturing. In this microbenchmark, we focus on the capture execution, which means that for systems like SQLProv and SmokedDuck who have two phases in their total capture phases, since we do not need to further utilize the provenance, we only examine how the systems perform in phase I. Besides the performance of each system, we also examine how the underlying database performs to execute queries.

T6: Provenance Size per Result Tuple. Fig. 8b shows the execution and storage cost for system to capture provenance for the two query templates.

¹¹Some versions of SmokedDuck do support backtracking provenance in the second phase of capture for individual result rows, but there limitations in the current implementation Sec. D.1

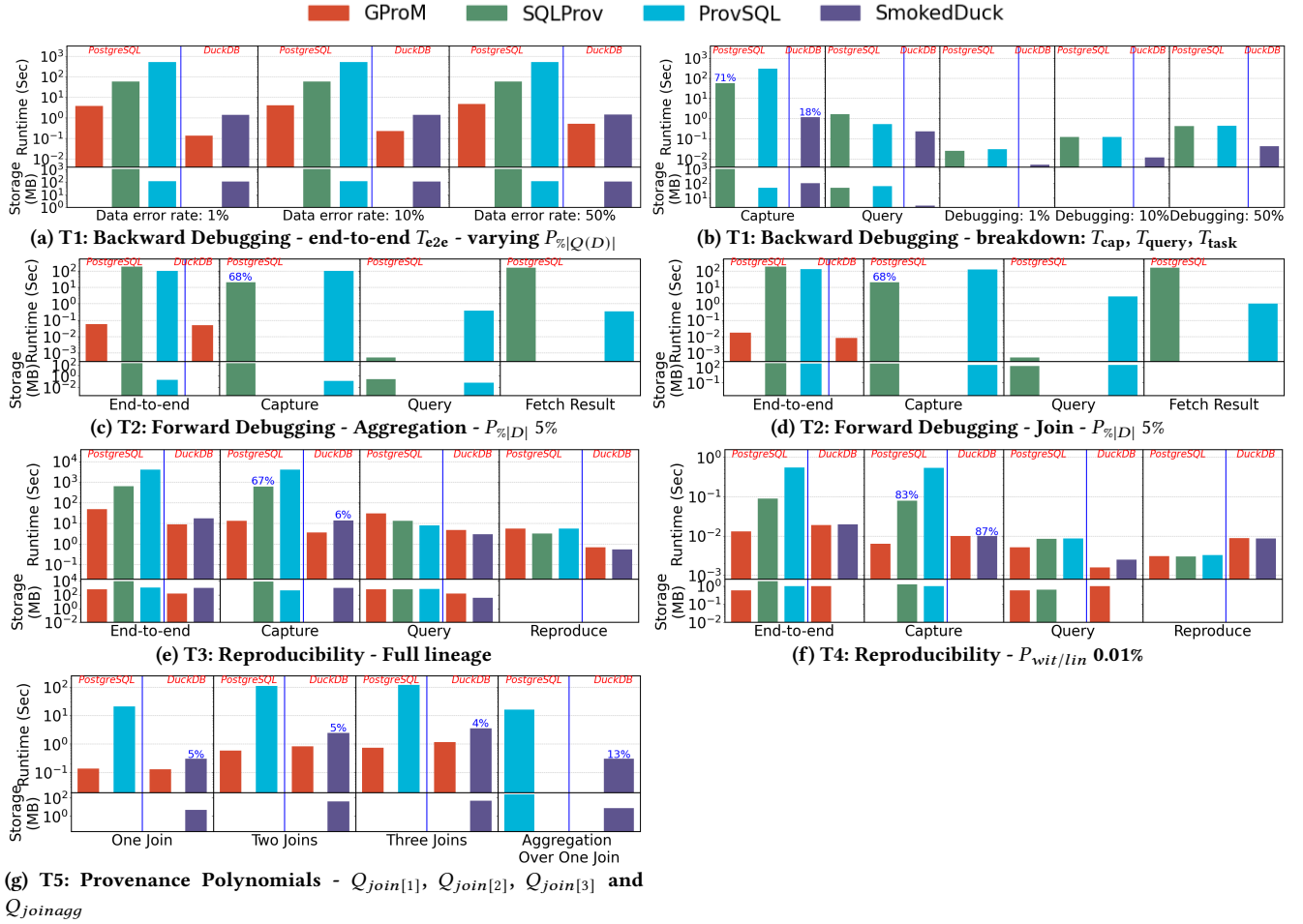


Figure 7: Experimental Evaluation for Use-case Specific Tasks

Both figures show a pattern: keeping the total provenance size the same, the more groups (distinct number of values) the query needs to process, the higher cost for a system to capture provenance. For SQLProv, phase I is expensive since it will log all input data of each operator to tables. The ProvSQL has the worst performance in both execution and storage since it should write all necessary data in memory map files and communicate with the data to construct the circuits for each result tuple. ProvSQL will produce same amount of data during capturing provenance for both query templates, as well as SmokedDuck, since the total provenance sizes are the same. While for SQLProv, it materializes different size of logging data for Q_{FPDist} query, since this system keeps only one arbitrary input tuple for each distinct value as the provenance.

T7: Varying Total Provenance Size. Fig. 8c and ?? show the experiment evaluation for systems capturing provenance.

Increasing the size of total provenance will increase the execution time to capture provenance, as well as the storage size of systems.

T8: Varying Intermediate Provenance Size.

Fig. 8d shows the experiment result of varying the having selectivity. The experiment shows that a larger selectivity will lead to higher cost in execution to capture provenance because more data

needs to be processed, but the storage used keeps the same since the total provenance size does not change.

??shows that both cost of execution and storage increase when there are more intermediate provenance data. For an individual system, the final result size keeps the same, while it has to process more intermediate provenance data to construct provenance for each result tuple.

T9: Provenance Polynomial Factorization. Fig. 8e shows the experimental evaluation for Q_{JA} and Q_{AJ} by fixing the filter in rate at 50%. We can observe that the cost to capture provenance for systems like GProM and SmokedDuck will decrease if first comes the group-by-aggregation since there are less data needed to join with the other table. For SQLProv, phase I cost is reduced (from 41.130246 seconds to 39.423197 seconds). This experiment show that even we keep fixed the sizes of total provenance and per result provenance, different provenance factorization leads to different performance of systems.

Fig. 8f demonstrates that vary the factorization of provenance can significantly affect the performance of systems to capture the provenance. Since GProM uses plain text representation, associating provenance combination with each result and displaying the times

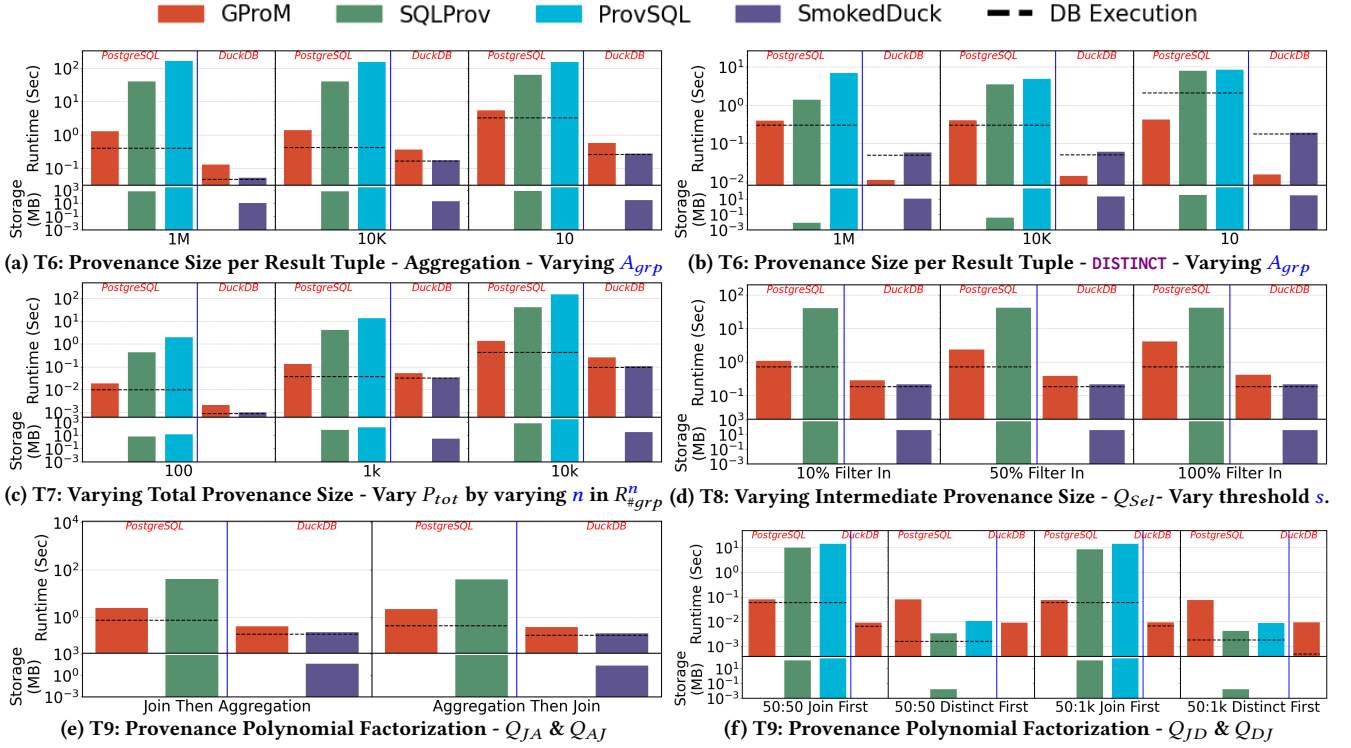


Figure 8: Experimental evaluation for Provenance Structure & Size Tasks

of number of combinations. The execution cost is the same even if varying the factorization. For SQLProv and ProvSQL, selection distinct value first can significantly reduce the cost to capture provenance since (1) this will keep only one copy of each distinct value, reducing the size of data participating the join operator, (2) this will reduce the size of data to be materialized, reducing time cost for SQLProv in phase II to construct provenance for query result, and reducing time for ProvSQL to communicate with mmap. Other experiments demonstrate that ProvSQL usually has the highest cost when capturing the provenance, while using **DISTINCT** operator to vary the provenance factorization can behave differently.

Insights: The experiments in this microbenchmark demonstrate that systems perform differently when varying different parameters in term of provenance result. For GProM, it joins the result data with provenance for each operator for the capture result, thus, the more size of provenance data it is, the higher the cost it is. When it comes the **DISTINCT** operator, the system outputs all input to this operator, without grouping distinct value, the performance is relative the same (Fig. 8a and Fig. 8b). GProM can have the best performance among evaluated systems that use PostgreSQL database. SQLProv and ProvSQL have higher cost compared to GProM. For SQLProv, besides computing result, each operator materializes its own provenance, which increases cost when capturing provenance. Therefore, the more provenance an operator has, the higher cost it is to complete the capturing process (??, ?? and ??). When it comes to factorization of provenance, computing provenance for each group (or distinct value) first leads to better

performance, since this can reduce the size of provenance data that participates in join operation. For ProvSQL, to compute the provenance, each operator writes the input tuples' tokens (UUID) into files on disk and this information serves as input for the next above operator when construct provenance for result, reading tokens information from disk. To build the provenance when the size of total provenance increases, the cost increase. When the provenance is fixed, the cost is relative stable (Fig. 8a and Fig. 8b), since the provenance tokens for each operator to process is the same. The factorization of provenance experiment demonstrates that ProvSQL can vary cost significantly to capture provenance when changing the factorization of per result tuple provenance. SmokedDuck has the best performance when evaluating provenance capturing using DuckDB. Unlike GProM that needs to join provenance data for result, which needs extra cost, SmokedDuck processes each operator and materialize necessary provenance information that is cheaper compare to GProM except experiment in Fig. 8a where GProM outputs all provenance for **DISTINCT** but GProM require computation for this operator.

Despite the performance of systems, each system has its own benefit. GProM outputs the result tuples along with the provenance tuples in one execution, which make it convenient to apply further operations on top of provenance data and this system require no extra storage. SQLProv and SmokedDuck complete capturing in two phases and each operator' provenance data is materialized inside database, which makes it easy to know details about provenance for operators. If users would like to know the

provenance of specific operators, SQLProv and SmokedDuck are the choices. ProvSQL has more formats in term of provenance representation: Why, Boolean and Counting sets representation and circuits for polynomials, which make users easy to examine provenance in different formats.

6.4 Query Specific Provenance Capture

T10: TPC-H. Sec. 6.4 shows provenance capturing for *TPCH* queries using the evaluated system in our work. The evaluation demonstrates (1) different capabilities of systems to support queries, and (2) different capture strategies exploited that result in different performance for same query.

T11: Top-K. To examine the performance of systems, we use a Top-K query template Q_{TopK} which applies top-k operator on top of group-by-aggregation operators. The total rows inputted into top-k operator is 1M. We vary the ratio of result tuples over total inputted tuples to 0.1%, 1% and 10%. The Fig. 9a presents the experiment evaluation of systems to capture provenance for this type of queries.

T13: multiagg In the experimental evaluation, we compare the performance of Q_{MLAgg} with Q_{FPAgg} , in which Q_{MLAgg} outputs the maximal values of two aggregate functions over all groups that computed in Q_{FPAgg} . The total number of groups is 1K. Fig. 9b presents the result of comparing one and two levels of aggregate functions.

Insights: The experiments in this section present systems' capability to capture provenance for different type of queries. Among all the evaluated systems, SQLProv can support a wider range of query types to capture provenance. ProvSQL does not support T11: Top-K, T19: Argmin and T12: Multi-Level Aggregation because the aggregation value is a specialized datatype in this system which cannot be used in further computation.

7 DISCUSSIONS

The PBench illustrates some important aspects that help in the development of a provenance system. First, systems should pay attention to the requirements of a use case task which can guide developers to optimize the core technology to capture provenance. Second, systems natively support more expressive models pay more for most use cases which requires simpler models. What is more, Systems use bottom-up strategy can suffer more intermediate provenance. Last but not the least, for two-phase capture systems, top-down strategy is more efficient when inferring the provenance due to less and targeted data needed in phase II.

8 RELATED WORK

Data provenance [15, 25, 32] traces the origin of data that contributes to the result of a query.

Provenance Models. Several provenance models have been proposed to provide a formal semantics for explaining query results under different classes of queries [15, 25, 32]. Why-provenance [11, 22] identifies which input tuples are used to generate the result. Where-provenance [11] traces the input tuples from which the result is derived. Why-not [14] model answers the missing results using provenance. [15, 25, 32, 35] present a detailed treatment of relational provenance models. Witness [11] denotes a subset of database

that is sufficient to generate the same result. Lineage [18–20, 22], capturing all inputs that contribute to a result of query, was first introduced in [61] and then was formalized based on sufficiency and necessity [22]. Green et al. [28, 29] introduced the K-relations and demonstrated the $\mathbb{N}[X]$ of provenance polynomials, where the elements are polynomials over variables X to represent tuples. Niu et al [50] proposes provenance sketches, which is a coarse-grained provenance model. Sketches capture provenance at the granularity of fragments, horizontally partitioned of a relation. The approach captures sketches over-approximating the provenance of a query and utilizes these sketches to speed-up subsequent queries. [2, 3] discussed the summarization of provenance polynomials.

Provenance Use Cases. There are some important use case tasks that provenance can conduct. Provenance enables the creation of "reproducibility packages" [16, 58]. While in database, reproducibility often means the sufficiency of data that guarantees to produce the same result [52, 53]. Provenance serves a tool for debugging. [49] proposes debugging of transactions with provenance. Provenance plays an essential role in probabilistic databases [57, 60], where the polynomial formulas provide symbolic representation of tuple derivations and can be used to compute the probability of query result tuples. In view maintenance under delete propagate scenario [12, 36], provenance polynomials assist How-to analysis [42, 43] to hypothetical reasoning the updated results.

Provenance Systems. Provenance can be captured by annotating data and propagating these annotations using relational queries or by extending the database system [37, 51]. Systems like GProM [6], Perm [27], Smoke [54], Smoked Duck [45], Links [24], ProvSQL[55] and DBNotes[8] capture provenance for SQL queries.

9 CONCLUSIONS AND FUTURE WORK

We present PBench, a comprehensive benchmark for relational provenance systems specifically designed to stress-test such systems, compare existing systems on a leveled playing field, and empower developers of such system to better understand the strengths and weaknesses of their systems. We use PBench to evaluate several SOTA relational provenance systems. The results of this evaluation unearthed

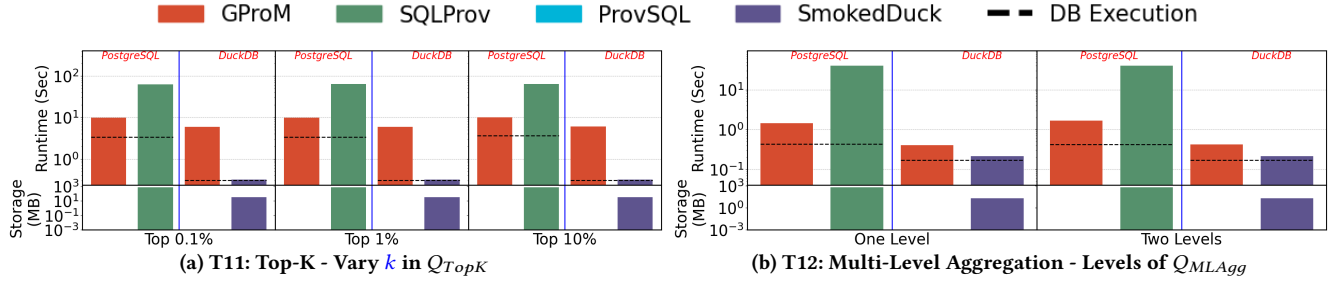


Figure 9: Experimental Evaluation for Query Constructs & Structure Tasks

Query	Database: PostgreSQL					Database: DuckDB				
	GProM Runtime	SQLProv Runtime (I)	SQLProv Runtime (II)	Storage	ProvSQL Runtime	ProvSQL Storage	GProM Runtime	SmokedDuck Runtime (I)	SmokedDuck Runtime (II)	Storage
Q_1	3.762148	25.63600	—	385.211	389.475503	8698.000 MB	0.843	0.1750	0.386625	9.50 MB
Q_2	—	0.148412	0.001978	0.046 MB	—	—	—	0.0156	0.017604	8.25 MB
Q_3	0.298262	0.852136	0.124622	3.062 MB	—	—	0.143	0.0734	0.022613	6.00 MB
Q_4	—	0.585017	0.291894	3.367 MB	—	—	—	0.0696	0.091305	4.75 MB
Q_5	0.342031	0.403161	0.075558	0.601 MB	—	—	0.185	0.0823	0.065690	5.50 MB
Q_6	0.340716	0.474468	—	—	0.199602	0.000 MB	0.0749	—	—	—
Q_7	1.601194	0.622116	0.037855	0.492 MB	0.593318	0.063 MB	0.337	0.1700	0.079125	26.25 MB
Q_8	0.210725	—	—	—	—	—	0.49	0.0585	0.054662	7.25 MB
Q_9	0.814160	3.087621	2.101858	25.750 M	4.03801	162.400 MB	2.4	0.1710	0.158642	8.25 MB
Q_{10}	0.438479	1.813259	0.410625	11.101 M	—	—	0.383	0.1370	0.015166	5.25 MB
Q_{11}	—	0.366711	0.005919	4.750 MB	—	—	—	—	—	—
Q_{12}	0.546049	—	—	—	0.713671	14.500 MB	0.135	0.0716	0.023840	3.75 MB
Q_{13}	1.779161	12.36488	69.966157	115.195 MB	—	—	0.644	0.1760	0.320564	13.25 MB
Q_{14}	1.144993	—	—	—	—	—	0.0757	—	—	—
Q_{15}	—	0.001716	0.000156	0.015 MB	—	—	—	—	—	—
Q_{16}	—	1.51182	0.230515	10.000 M	—	—	—	0.0237	0.033327	2.00 MB
Q_{17}	—	4.198612	0.004298	12.226 M	—	—	—	—	—	—
Q_{18}	—	1.427144	0.001401	0.054 MB	—	—	—	0.3750	0.498652	36.50 MB
Q_{19}	0.036617	0.04997	0.001122	0.023 MB	0.023681	0.000 MB	0.157	—	—	—
Q_{20}	—	—	—	—	—	—	—	0.0668	0.028663	8.00 MB
Q_{21}	—	0.91498	0.02119	0.373 MB	—	—	—	0.1850	0.310530	11.50 MB
Q_{22}	—	0.098775	0.008513	0.437 MB	—	—	—	—	—	—

Figure 10: T10: TPC-H – Execution Time and Storage Overhead

REFERENCES

- [1] 2007. 2nd Provenance Challenge. <https://openprovenance.org/provenance-challenge/SecondProvenanceChallenge.html>.
- [2] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2015. Approximated Summarization of Data Provenance. In *CIKM*, James Bailey, Alistair Moffat, Charu C. Aggarwal, Maarten de Rijke, Ravi Kumar, Vanessa Murdock, Timos K. Sellis, and Jeffrey Xu Yu (Eds.). 483–492.
- [3] Eleanor Ainy, Pierre Bourhis, Susan B. Davidson, Daniel Deutch, and Tova Milo. 2016. PROX: Approximated Summarization of Data Provenance. In *EDBT*, Evaggelia Pitoura, Sofian Maabout, Georgia Koutrika, Amélie Marian, Letizia Tanca, Ioana Manolescu, and Kostas Stefanidis (Eds.). 620–623.
- [4] Shatha Algarni, Boris Glavic, Seokki Lee, and Adriane Chapman. 2025. Efficient Query Repair for Aggregate Constraints. *Proceedings of the VLDB Endowment* (2025).
- [5] Yael Amsterdamer, Daniel Deutch, and Val Tannen. 2011. Provenance for Aggregate Queries. In *PODS*. 153–164.
- [6] Bahareh Arab, Su Feng, Boris Glavic, Seokki Lee, Xing Niu, and Qitian Zeng. 2018. GProM - A Swiss Army Knife for Your Provenance Needs. *IEEE Data Eng. Bull.* 41, 1 (2018), 51–62.
- [7] Bahareh Arab, Dieter Gawlick, Vasudha Krishnaswamy, Venkatesh Radhakrishnan, and Boris Glavic. 2018. Using Reenactment to Retroactively Capture Provenance for Transactions. *TKDE* 30, 3 (2018), 599–612.
- [8] Deepavali Bhagwat, Laura Chiticariu, Wang Chiew Tan, and Gaurav Vijayvargiya. 2005. An annotation management system for relational databases. *VLDBJ* 14, 4 (2005), 373–396.
- [9] Nicole Bidoit, Melanie Herschel, and Aikaterini Tzompanaki. 2015. Efficient Computation of Polynomial Explanations of Why-Not Questions. In *CIKM*, James Bailey, Alistair Moffat, Charu C. Aggarwal, Maarten de Rijke, Ravi Kumar, Vanessa Murdock, Timos K. Sellis, and Jeffrey Xu Yu (Eds.). 713–722.
- [10] Nicole Bidoit, Melanie Herschel, Katerina Tzompanaki, et al. 2014. Query-Based Why-Not Provenance with NedExplain. In *EDBT*.
- [11] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330.
- [12] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2002. On Propagation of Deletions and Annotations through Views. In *PODS*. 150–158.
- [13] Felix Campbell, Bahareh Arab, and Boris Glavic. 2022. Efficient Answering of Historical What-if Queries. In *SIGMOD*. 1556–1569.
- [14] Adriane Chapman and H. V. Jagadish. 2009. Why Not?. In *SIGMOD*. 523–534.
- [15] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474.
- [16] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. 2016. Reprozip: Computational reproducibility with ease. In *Proceedings of the 2016 international conference on management of data*. 2085–2088.
- [17] Gao Cong, Wenfei Fan, Floris Geerts, Jianzhong Li, and Jizhou Luo. 2012. On the Complexity of Annotation Propagation and View Update Analyses. *IEEE TKDE* 24, 3 (2012), 506–519.
- [18] Yingwei Cui and Jennifer Widom. 2000. Lineage Tracing in a Data Warehousing System. In *ICDE*. 683–684.
- [19] Yingwei Cui and Jennifer Widom. 2000. Practical Lineage Tracing in Data Warehouses. In *ICDE*. 367–378.
- [20] Yingwei Cui and Jennifer Widom. 2001. Lineage Tracing for General Data Warehouse Transformations. In *VLDB*. 471–480.
- [21] Yingwei Cui and Jennifer Widom. 2001. *Run-time translation of view tuple deletions using data lineage*. Technical Report. Stanford.
- [22] Yingwei Cui, Jennifer Widom, and Janet L. Wiener. 2000. Tracing the Lineage of View Data in a Warehousing Environment. *TODS* 25, 2 (2000), 179–227.
- [23] Ralf Diestelkämper, Seokki Lee, Melanie Herschel, and Boris Glavic. 2021. To not miss the forest for the trees - A holistic approach for explaining missing answers over nested data. In *SIGMOD*. 405–417.
- [24] Stefan Fehrenbach and James Cheney. 2018. Language-integrated provenance. *Sci. Comput. Program.* 155 (2018), 103–145.
- [25] Boris Glavic. 2021. Data Provenance - Origins, Applications, Algorithms, and Models. *Foundations and Trends® in Databases* 9, 3-4 (2021), 209–441.
- [26] Boris Glavic, Pengyuan Li, Ziyu Liu, Dieter Gawlick, Vasudha Krishnaswamy, Danica Porobic, and Zhen Hua Liu. 2024. Towards an Objective Metric for Data Value Through Relevance. In *Proceedings of the 14th Conference on Innovative Data Systems*.
- [27] Boris Glavic, Renée J. Miller, and Gustavo Alonso. 2013. Using SQL for Efficient Generation and Querying of Provenance Information. In *In Search of Elegance in the Theory and Practice of Computation - Essays Dedicated to Peter Buneman, Val Tannen, Limsoon Wong, Leonid Libkin, Wenfei Fan, Wang-Chiew Tan, and Michael P. Fourman* (Eds.), Vol. 8000. 291–320.
- [28] Todd J. Green, Gregory Karvounarakis, and Val Tannen. 2007. Provenance Semirings. In *PODS*. 31–40.
- [29] Todd J Green and Val Tannen. 2017. The Semiring Framework for Database Provenance. In *PODS*. 93–99.
- [30] Erich Grädel and Val Tannen. 2017. Semiring Provenance for First-Order Model Checking. *arXiv preprint arXiv:1712.01980* (2017).
- [31] Erich Grädel and Val Tannen. 2020. Provenance analysis for logic and games. *Moscow Journal of Combinatorics and Number Theory* 9, 3 (2020), 203–228.
- [32] Melanie Herschel, Ralf Diestelkämper, and Houssem Ben Lahmar. 2017. A survey on provenance: What for? What form? What from? *VLDBJ* (2017), 1–26.
- [33] Rinke Hoekstra and Paul Groth. 2014. PROV-O-Viz-understanding the role of activities in provenance. In *International Provenance and Annotation Workshop*. Springer, 215–220.
- [34] Xiao Hu and Stavros Sintos. 2024. Finding Smallest Witnesses for Conjunctive Queries. In *ICDT*.
- [35] G. Karvounarakis and T.J. Green. 2012. Semiring-Annotated Data: Queries and Provenance. *SIGMOD Record* 41, 3 (2012), 5–14.
- [36] Benny Kimelfeld, Jan Vondrák, and Ryan Williams. 2012. Maximizing Conjunctive Views in Deletion Propagation. *TODS* 37, 4 (2012), 24:1–24:37.
- [37] Sven Köhler, Bertram Ludäscher, and Yannis Smaragdakis. 2012. Declarative datalog debugging for mere mortals. In *International Datalog 2.0 Workshop*. Springer, 111–122.
- [38] Seokki Lee, Sven Köhler, Bertram Ludäscher, and Boris Glavic. 2016. Implementing Unified Why- and Why-Not Provenance Through Games. In *TaPP*.
- [39] Seokki Lee, Bertram Ludäscher, and Boris Glavic. 2018. PUG: a framework and practical implementation for why and why-not provenance. *VLDBJ* 28, 1 (2018), 47–71.
- [40] Jinyang Li, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 17, 2 (2023), 106–118.

- [41] Jinyang Li, Alon Silberstein, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. ERICA: Query Refinement for Diversity Constraint Satisfaction. *PVLDB* 16, 12 (2023), 4070–4073.
- [42] Alexandra Meliou, Wolfgang Gatterbauer, and Dan Suciu. 2011. Reverse data management. *PVLDB* 4, 12 (2011).
- [43] A. Meliou and D. Suciu. 2012. Tiresias: The database oracle for how-to queries. In *SIGMOD*. 337–348.
- [44] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT*. 862–873.
- [45] Haneen Mohammed, Charlie Summers, Sugghosh V. Kaushik, and Eugene Wu. 2023. SmokedDuck Demonstration: SQLStepper. In *SIGMOD*, Sudipto Das, Ipokratis Pandis, K. Selçuk Candan, and Sihem Amer-Yahia (Eds.). 183–186.
- [46] Haneen Mohammed, Eugene Wu, Alexander Yao, Charlie Summers, Lampros Flokas, Gromit Yeuk-Yin Chan, Subrata Mitra, and Hongbin Zhong. 2024. Fade: More Than a Million What-Ifs Per Second. *PVLDB* 18, 4 (2024), 943–955.
- [47] L. Moreau, B. Ludascher, I. Altintas, R.S. Barga, S. Bowers, S. Callahan, G. Chin Jr, B. Clifford, S. Cohen, S. Cohen-Boulakia, et al. 2007. The First Provenance Challenge. *Concurrency and Computation: Practice and Experience* (2007).
- [48] Tobias Müller, Benjamin Dietrich, and Torsten Grust. 2018. You say ‘what’, i hear ‘where’ and ‘why’: (mis-) interpreting SQL to derive fine-grained provenance. *PVLDB* 11, 11 (2018), 1536–1549.
- [49] Xing Niu, Boris Glavic, Seokki Lee, Bahareh Arab, Dieter Gawlick, Zhen Hua Liu, Vasudha Krishnaswamy, Su Feng, and Xun Zou. 2017. Debugging Transactions and Tracking their Provenance with Reenactment. *PVLDB* 10, 12 (2017), 1857–1860.
- [50] Xing Niu, Boris Glavic, Ziyu Liu, Pengyuan Li, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, and Danica Porobic. 2021. Provenance-based Data Skipping. *PVLDB* 15, 3 (2021), 451–464.
- [51] Xing Niu, Raghav Kapoor, Boris Glavic, Dieter Gawlick, Zhen Hua Liu, and Venkatesh Radhakrishnan. 2017. Provenance-Aware Query Optimization. In *ICDE*. 473–484.
- [52] Quan Pham, Tanu Malik, Boris Glavic, and Ian Foster. 2015. LDV: Light-weight Database Virtualization. In *ICDE*. 1179–1190.
- [53] Quan Pham, Richard Whaling, Boris Glavic, and Tanu Malik. 2015. Sharing and Reproducing Database Applications. *PVLDB* 8, 12 (2015), 1988–1999.
- [54] Fotis Psallidas and Eugene Wu. 2018. Smoke: Fine-grained Lineage at Interactive Speed. *PVLDB* 11, 6 (2018), 719–732.
- [55] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. ProvSQL: Provenance and Probability Management in PostgreSQL. *PVLDB* 11, 12 (2018), 2034–2037.
- [56] Pierre Senellart, Louis Jachiet, Silviu Maniu, and Yann Ramusat. 2018. ProvSQL: provenance and probability management in postgresql. *PVLDB* 11, 12 (2018), 2034–2037.
- [57] Dan Suciu. 2020. Probabilistic databases for all. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 19–31.
- [58] Dai Hai Ton That, Gabriel Fils, Zhihao Yuan, and Tanu Malik. 2017. Sciunits: Reusable research objects. *arXiv preprint arXiv:1707.05731* (2017).
- [59] Quoc Trung Tran and Chee-Yong Chan. 2010. How to ConQueR why-not questions. In *SIGMOD*. 15–26.
- [60] Guy Van den Broeck, Dan Suciu, et al. 2017. Query processing on probabilistic data: A survey. *Foundations and Trends® in Databases* 7, 3-4 (2017), 197–341.
- [61] Allison Woodruff and Michael Stonebraker. 1997. Supporting Fine-grained Data Lineage in a Database Visualization Environment. In *ICDE*. 91–102.
- [62] Eugene Wu, Fotis Psallidas, Zhengjie Miao, Haoci Zhang, and Laura Rettig. 2017. Combining Design and Performance in a Data Visualization Management System. In *CIDR*.
- [63] Jane Xu, Waley Zhang, Abdussalam Alawini, and Val Tannen. 2018. Provenance Analysis for Missing Answers and Integrity Repairs. *Data Engineering* (2018), 39.

A ADDITIONAL DETAILS - QUERY TEMPLATES

A.1 Use-case Specific Tasks

A.1.1 Backward Debugging.

Q_{BDebug} is used in T1: Backward Debugging, shown below:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM  $R_{grp}^{1k}$  JOIN  $R_{join}^c$  ON (ga = c1to10)
GROUP BY ga
```

A.1.2 Forward Debugging.

Q_{FD1} is used in T15: Forward Debugging and T2: Forward Debugging, shown below:

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM  $R_{grp}^{1k}$ ,  $R_{join}^c$ 
WHERE ga = c1to10 and ga >= 1 and ga <= 333
GROUP BY ga
```

Q_{FD2} is used in T2: Forward Debugging and T16: Forward Debugging, shown below:

```
SELECT ga, va, vb
FROM  $R_{grp}^{1k}$ ,  $R_{join}^c$ 
WHERE ga = c1to10
AND ga >= 667
AND ga <= 1000
```

A.1.3 Reproducibility.

Q_{Lin} is used in T3: Reproducibility, shown below:

```
SELECT gb, min(va) AS minva
FROM  $R_{fp}^{10M}$  JOIN  $R_{join}^c$  ON gb = c1to10
GROUP BY gb
```

Q_{Wit} is used in T4: Reproducibility, shown below:

```
SELECT gb, min(va) AS minva
FROM  $R_{fp}^n$  JOIN  $R_{join}^c$  ON gb = c1to10
GROUP BY gb
```

Q_{AP} is used in T17: Reproducibility, shown below:

```
SELECT gb, avg(va) AS ava
FROM  $R_{fp}^{10M}$  JOIN  $R_{join}^c$  ON gb = c1to10
GROUP BY gb HAVING ava < 15
```

A.1.4 Provenance Polynomials.

The three SPJ queries, $Q_{join[1]}$, $Q_{join[2]}$, $Q_{join[3]}$, varying the number of join operations, and one SPJA query, $Q_{joinagg}$ are used for T5: Provenance Polynomials, show as below respectively:

```
SELECT ga
FROM  $R_{grp}^{1k}$  JOIN  $R_{join}^s$  ON ga = p10
```

```
SELECT ga
FROM  $R_{grp}^{1k}$  JOIN  $R_{join}^s$  ON ga = p10 JOIN  $R_{join}^j$  ON ga = c5
```

```
SELECT ga
FROM  $R_{grp}^{1k}$  JOIN  $R_{join}^s$  ON ga = p10
JOIN  $R_{join}^j$  ON ga = c5
JOIN  $R_{join}^{jj}$  ON ga = cc1
```

```
SELECT ga, avg(va) AS ava
FROM  $R_{grp}^{1k}$  JOIN  $R_{join}^s$  ON ga = p10
GROUP BY ga
```

A.2 Provenance Structure & Size

A.2.1 Provenance Size per Result Tuple.

Q_{FPagg} and Q_{FPdist} are used for T6: Provenance Size per Result Tuple, show as below respectively:

```
SELECT  $A_{grp}$ , avg(va) AS ava, avg(vb) AS avb
FROM  $R_{fp}^{10M}$ 
GROUP BY  $A_{grp}$ 
```

```
SELECT DISTINCT  $A_{grp}$ 
FROM  $R_{fp}^{10M}$ 
```

A.2.2 Varying Total Provenance Size.

Q_{VPGN} and Q_{VPGS} , show as below respectively, are used in T7: Varying Total Provenance Size to vary the total provenance size. Q_{VPGN} (Q_{VPGS}) fixes the number of groups n to $1k$ (number of tuples per group n to $1k$) and vary the number of tuples per group (number of groups) to $\{100, 1K, 10K\} \cdot P_{SF}$.

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grpn
GROUP BY ga
```

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM Rgrpsizen
GROUP BY ga
```

A.2.3 Intermediate Provenance Size.

Q_{Sel} and $Q_{Join\#1}$, $Q_{Join\#2}$ and $Q_{Join\#3}$ (shown below respectively) are used for T8: Varying Intermediate Provenance Size.

```
SELECT gb, avg(va) AS ava, avg(vb) AS avb
FROM Rfp10M JOIN Rjoinc ON gb = c1to1
GROUP BY gb HAVING avg(va) < s
```

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grp1k, Rjoins
WHERE ga = p10
GROUP BY ga
```

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grp1k, Rjoins, Rjoinj
WHERE ga = p10 AND ga = c5
GROUP BY ga
```

```
SELECT ga, avg(va) AS ava, avg(vb) AS avb
FROM R#grp1k, Rjoins, Rjoinj
WHERE ga = p10 AND ga = c5 AND ga = cc2
GROUP BY ga
```

A.2.4 Provenance Factorization. Q_{DJ}

```
SELECT t1.da
FROM dj1 t1 JOIN dj2 t2
ON t1.da = t2.da;
```

Q_{JD} used in T9: Provenance Polynomial Factorization is a **DISTINCT** on top of a join. As mentioned in Sec. 5.5.4, we control P_{fact} (how factorizable the provenance is) by select the result attribute A and the attribute from S we join on (B).

```
SELECT DISTINCT R.A
FROM dj1 AS R JOIN dj2 AS S ON (R.A = S.B)
```

Query Q_{DJ} additionally eliminated duplicates before joining R and S :

```
SELECT DISTINCT R.A
from (SELECT DISTINCT A FROM dj1) AS R
JOIN
(SELECT DISTINCT A FROM dj2) AS S
on R.A = S.B;
```

Note that the two joined tables have the same schema and data distribution, simulating a self-join. Each table has $10K \cdot P_{SF}$ rows. Both A and B are chosen from one of three attributes: da (one distinct value), db (50 distinct values) and dc ($1k$ distinct values). We evaluate each pair of these three attributes from two tables. For first query Q_{JD} which yields a polynomial of the form $\sum r_i \times s_i$, P_{fact} is 1 independent of the choice of A and B as the provenance

δ	Join Condition	#P ($\sum_i r_i \cdot (\sum_j s_j)$)	#SOP(P) ($\sum r_i \times s_i$)	P_{fact}
da	R.da = S.da	$2 \cdot 10K \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 10K \cdot P_{SF}$	$\frac{1}{10K \cdot P_{SF}}$
	R.da = S.db	$(10K + 200) \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 200 \cdot P_{SF}$	$\sim \frac{1}{400 \cdot P_{SF}}$
	R.da = S.dc	$(10K + 10) \cdot P_{SF}$	$2 \cdot 10K \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\sim \frac{1}{20 \cdot P_{SF}}$
db	R.db = S.db	$2 \cdot 200 \cdot P_{SF}$	$2 \cdot 200 \cdot P_{SF} \cdot 200 \cdot P_{SF}$	$\frac{1}{200 \cdot P_{SF}}$
	R.db = S.dc	$(200 + 10) \cdot P_{SF}$	$2 \cdot 100 \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\sim \frac{1}{20 \cdot P_{SF}}$
dc	R.dc = S.dc	$(10 + 10) \cdot P_{SF}$	$2 \cdot 10 \cdot P_{SF} \cdot 10 \cdot P_{SF}$	$\frac{1}{10 \cdot P_{SF}}$

Figure 11: **DISTINCT** P_{fact}

for every result tuple is already expressed as a flat sum-of-products. Fig. 11 show P_{fact} for the second query.

Additionally, in this task we use Q_{JA} , an aggregation on top of a join. We vary selectivity ($\{10\%, 50\%, 100\%\}$) by varying the threshold s

```
SELECT gb, avg(va) AS ava, avg(vb) AS avb
FROM Rfp10M AS R JOIN Rjoinc AS S ON gb = c1to1
GROUP BY gb
HAVING avg(va) < s
```

The factorized version, Q_{AJ} , pushes the aggregation into the left side of the join, yielding factorized provenance:

```
SELECT gb, ava, avb
FROM (
  SELECT gb, avg(va) AS ava, avg(vb) AS avb
  FROM Rfp10M GROUP BY gb HAVING avg(va) < s
) AS R JOIN Rjoinc AS S ON gb = c1to1
```

Query Q_{JA} produces a flat sum-of-products $\delta(\sum r_i \times s_i)$ and Q_{AJ} returns factorized polynomials of the form $\delta((\sum_i r_i) \cdot s_i)$. The number of groups is $1K \cdot P_{SF}$ and each group has $10K \cdot P_{SF}$ tuples. For the first aggregation query we have $P_{fact} = 1$. For the factorized query, $P_{fact} \sim \frac{1}{2}(\frac{10K+1}{10K \cdot 2})$.

A.3 Query Constructs & Structure

A.3.1 Top-K.

Q_{TopK} , shown below, is used for T14: **topk**

```
SELECT gc, avg(va) AS ava, avg(vc) AS avc
FROM Rfp10M
GROUP BY gc
ORDER BY ava
LIMIT 100
```

A.3.2 Multi-Level Aggregation.

Q_{MLAgg} is used for T12: Multi-Level Aggregation shown below.

```
SELECT max(va) AS maxava, max(vb) AS maxvb
FROM (SELECT gb, avg(va) AS va, avg(vb) AS vb
FROM Rfp10M GROUP BY gb) AS sub
```

B ADDITIONAL BENCHMARK TASK DESCRIPTIONS

B.1 Use-case Specific Tasks

B.1.1 Forward Debugging. In T15: **Forward debug - impacted queries** the system is provided with a set of queries and a set of rows from the database that are considered to be erroneous. This task is then to determine for each query Q in the set whether it has at least one of the erroneous rows in its provenance, i.e., the query's result is potentially affected by the error. We use a query template, Q_{FD1} (shown below),

```

SELECT ga, avg(va) AS avb, avg(vb) AS avb
FROM R#grp1k, Rjoinc
WHERE ga = c1to10 and ga >= 1 and ga <= 333
GROUP BY ga

```

working on two synthetic tables: $R_{\#grp}^{1k}$ and R_{join}^c . We vary the error data range in table $R_{\#grp}^{1k}$ $P_{\%|D|}$ in 1%, 10% and 30%. The last task **T16: Forward debugging - provenance and result** maps the erroneous provenance tuples with the affected result tuples, and utilizes the same data and query settings as T2.

B.1.2 Reproducibility. T17: Reeval. - coarse-grained uses coarse-grained provenance. For this task we consider the scenario of using provenance captured for one query to answer another future query [50, 62]. In this context, coarse-grained provenance has the advantage of small storage size. We use a query Q_{AP} (shown below), and vary the number of fragments, rows of table R_{fp}^{10M} are grouped into, which determines the granularity of the captured provenance $P_{\#frag} \in \{10, 100, 500\}$.

```

SELECT gb, avg(va) AS avb
FROM Rfp10M JOIN Rjoinc on gb = c1to10
GROUP BY gb HAVING avb < 15

```

As in the reproducibility tasks, the coarse-grained provenance has to be captured and then in a separate step is used to answer a query.

B.2 Provenance Structure & Size Tasks

In this part, we provide additional design of varying structure of queries and what query constructs are used in PBench.

B.2.1 Complex Join Conditions. T18: Complex Join uses a join query with a more complex join condition expressed as a disjunctive **WHERE** condition followed by an aggregation. These complex conditions prevent database engines from exploiting the standard optimizations, such as selection-push-down. Thus, this creates a computationally intensive environment that forces the systems to fully evaluate the join predicates during the capture process. We use a query Q_{CplxJ} (shown below) which joins tables $R_{\#grp}^{1k}$ and R_{join}^c .

```

SELECT ga, count(vb) AS cvb, avg(vb) AS avb
FROM R#grp1k, Rjoinc
WHERE
  (ga >= 0 AND ga <= 300
   AND va >= 20 AND va <= 280
   AND vb >= 150 and vb <= 250
   AND ga = c1to50)
OR
  (ga >= 301 AND ga <= 600
   AND va >= 320 AND va <= 580
   AND vb >= 450 AND vb <= 550
   AND ga = c1to50)
OR
  (ga >= 601 AND ga <= 900
   AND va >= 620 AND va <= 880
   AND vb >= 750 AND vb <= 850
   AND ga = c1to50)
GROUP BY ga

```

B.2.2 Argmin. T19: Argmin is the argmin query Q_{AMin} (shown below) which is a one-to-one join query that computes the minimum for a group and then joins the aggregation result back to the input table to retrieve additional attributes from the row(s) with the

minimum value. This query runs over two tables, $R_{\#grp}^{1k}$ and R_{join}^c , and yields a factorized polynomial $(\sum_i r_i) \cdot s_i ((\sum_{1K \cdot P_{SF}} r_i) \cdot 1)$.

```

SELECT ga, minv, c1to10
FROM (SELECT ga, min(va) AS minv
      FROM R#grp1k
      GROUP BY ga) AS R
JOIN Rjoinc AS S ON R.minv = S.c1to10

```

B.2.3 Recursive. T20: Recursive Query uses a recursive query Q_{RCR} (shown below) that returns the end points and lengths of paths in a graph (relation R_{graph}) that have less than d steps. We vary parameter $d \in \{2, 3\}$. The R_{graph} represents a dense random graph where each node has an average out degree of 100. Thus, Increasing the d will lead to an exponential expansion of search space. The size of the provenance for this query grows significantly in d .

```

WITH RECURSIVE rcr AS (
  SELECT f, t, 1 AS d
  FROM Rgraph
  WHERE f <= 10
  UNION ALL
  SELECT p.f AS f, p.t AS t, r.d + 1 AS d
  FROM rcr as r, Rgraph as p
  WHERE r.t = p.f and r.d < d
)
SELECT * FROM rcr;

```

B.2.4 Window Function. T21: Window Func. evaluates provenance capture for queries containing window functions. Window functions can result in large provenance. For instance, when using windows with **UNBOUNDED PRECEDING** the provenance of a row contains all rows that precede this row in sort order. However, such provenance is highly compressible as the provenance of consecutive rows overlap to a large degree. We use two variants of query Q_{Win} (shown below) accessing table $R_{\#grp}^{1k}$: the first uses **PARTITION BY** resulting in provenance of limited size per result row while the second omits the **PARTITION BY** clause, resulting in large provenance (on average half the input table is in the provenance of a result tuple).

```

SELECT ga, avg(va) OVER ([PARTITION BY ga]
                        ORDER BY id) AS rkava
FROM R#grp1k;

```

C ADDITIONAL EXPERIMENTAL RESULTS

C.1 Use-case Specific Tasks

T2: Forward Debugging. Fig. 7c and show the end-to-end and breakdowns evaluation for templates Q_{FD1} and Q_{FD2} , respectively. Note that SmokedDuck does not support for this kind of debugging since it uses the physical row id to trace provenance and there is a no general approach to find the connection of identifiers between the result tuples and provenance tuples.

T2: Forward Debugging. Fig. 7d shows show the end-to-end and breakdowns evaluation for templates Q_{FD1} and Q_{FD2} , respectively. Note that SmokedDuck does not support for this kind of debugging since it uses the physical row id to trace provenance and there is a no general approach to find the connection of identifiers between the result tuples and provenance tuples.

T16: Forward Debugging. For debugging, lineage serves as a tool to trace the error original data and query and result. The above

debugging experiment focus on data or result solely, in this experiment, we focus on the combination of lineage and result together. When an error is detected in the result or in the original data, it is interested to know which lineage data cause what error result. Thus, mapping lineage to result is needed in this case. ?? and ?? present the evaluation for this use case Q_{FD1} and Q_{FD2} .

T4: Reproducibility. ?? show the experimental result of query Q_{Lin} for reproducibility using provenance models lineage, minimal witness and 20% lineage. These figures shows (1) the end-to-end performance of all systems, and (2) performance breakdowns of systems SQLProv, ProvSQL and SmokedDuck. ProvSQL needs to read from and write to memory mapped files, and thus this leads worst performance among all systems. The minimal witness model requires least cost of time and storage to complete the reproducibility tasks, because the amount of total provenance data becomes much less compared to lineage model which has the worst performance. For SmokedDuck and SQLProv, the percentage of phase I over total capture time increases when size of lineage decreases, because (1) phase II is more sensitive to the size of materialized data, and less lineage means less computation in phase II, and (2) phase I is a must-have phase even if a query produce no results. In terms of storage usage, lineage model is obviously largest. Note that for minimal witness, SmokedDuck consumes storage but data size is less than one block, and thus, actual value cannot be queried. Minimal witness model is an optimized for reproducing the result (reproduce cost) in terms of both execution and storage cost.

T17: Reproducibility. ?? presents the GProM’s capability to capture the coarse-grained provenance. Compared to the table fp partitioned 10 ranges, number of ranges of 100 and 500 lead to higher cost in capture the provenance but less cost in reuse the sketches. Thus, the finer-grained partitioning leads to skip more irrelevant data to achieve faster query processing.

C.2 Provenance Structure & Size

C.3 Query Constructs & Structure

T18: Complex Join Conditions. ?? shows the evaluation of systems to capture provenance for the query Q_{CplxJ} .

T19: Argmin. ?? presents the experimental evaluation of capturing provenance for provenance systems using query Q_{AMin} .

T20: Recursive. To examine the performance of SQLProv to capturing provenance over recursive queries, we use data that simulating the graph traversal. The total nodes in the network are 100K in $P_{SF}=1$. The query template Q_{RCR} trace the direct and immediate neighbors for given starting node. In our experiment, we control the depth in 2 and 3 to detect the neighbor nodes. ?? shows the evaluation of SQLProv capturing provenance for the recursive query. The complexity of a path (number of nodes in a path) determines the cost of the performance.

T21: Window Function. We use a query template Q_{Win} which contains a window function. By varying the behavior of how windows are construct with or without partition function, we check the performance of SQLProv. ?? presents the evaluation for SQLProv. The evaluation shows that more windows in the result will increase the cost of a system to capture the performance.

D ADDITIONAL DETAILS

D.1 Provenance System Setup and Evaluation Protocol

D.1.1 GProM.

D.1.2 ProvSQL. ProvSQL is available on github: <https://github.com/PierreSenellart/provsql>. The system is constructed as a Postgres extension. We build the extension based on git commit .

D.1.3 SQLProv. The full code of SQLProv is not publicly available. We reached out to the authors and were provided with code and instructions that enabled us to run their approach.

D.1.4 SmokedDuck. SmokedDuck is available on github (<https://github.com/cudbg/sd>) and over pipy (<https://pypi.org/project/smokedduck/>). However, the version on pipy crashes with a segfault. We were in contact with the authors and based on their feedback did use the code available in the branch using git commit . The different branches of SmokedDuck differ in how backtracking in phase II operates. Some versions provide an interface where the user provides a single output rowid and the system traces back provenance for that row only while others construct a query that returns the full provenance. The branch we utilized was using the single rowid backtracking. While we were able to run phase I successfully with this version of SmokedDuck, we encountered issues with phase II. With feedback from the authors we were able to get but this failed for many queries in the benchmark. SmokedDuck is a forked version of duckdb based on duckdb version 1.0.0. For fairness of comparison we used this version in the evaluation for GProM too.

D.2 Join Selectivity

We now discuss the selectivity of joins used in queries in the benchmark. Consider a star join query, where R_0 is the center of the star, i.e., R_0 joins with each R_i .

```
SELECT ...
FROM R0, R1, ..., Rn
WHERE ...
```

We use an unified notation for join query selectivity as *selectivity profiles* that compactly describe the selectivity in a query with multiple joins. The *selectivity profile* of such an n -way join is of the form $x(1[y] - j_1 - \dots - j_n)$. Here, $x\%$ is the join result size relative to the size of the left-most table R_0 , e.g., $x = 2.0$ would imply that the result of the join is twice the size of relation R_0 . Parameter $y\%$ is the fraction of tuples from the left-most table R_0 that have join partners with the result of joining all remaining tables. Furthermore, j_i is the average number of tuples from the $i + 1^{th}$ table that can join with a tuple from R_0 . For example, $100\%(1[10\%] - 10)$, is a query with one join whose output is the same size as R_0 due roughly 10% of tuples from R_0 having join partners, but each such tuple on average has 10 join partners in R_1 . As another example, consider $500\%(1[10\%] - 10 - 5)$ which describes a three-way join query which returns $5 \cdot |R_0|$ tuples, because on average every of the 10% tuples from R_0 that have join partners in both R_1 and R_2 do join with 10 tuples in R_1 and 5 tuples in R_2 .

D.2.1 Selectivity Profiles for Benchmark Queries. $100\%(1[10\%] - 10)$ $500\%(1[10\%] - 10 - 5)$ $1000\%(1[10\%] - 10 - 5 - 2)$.

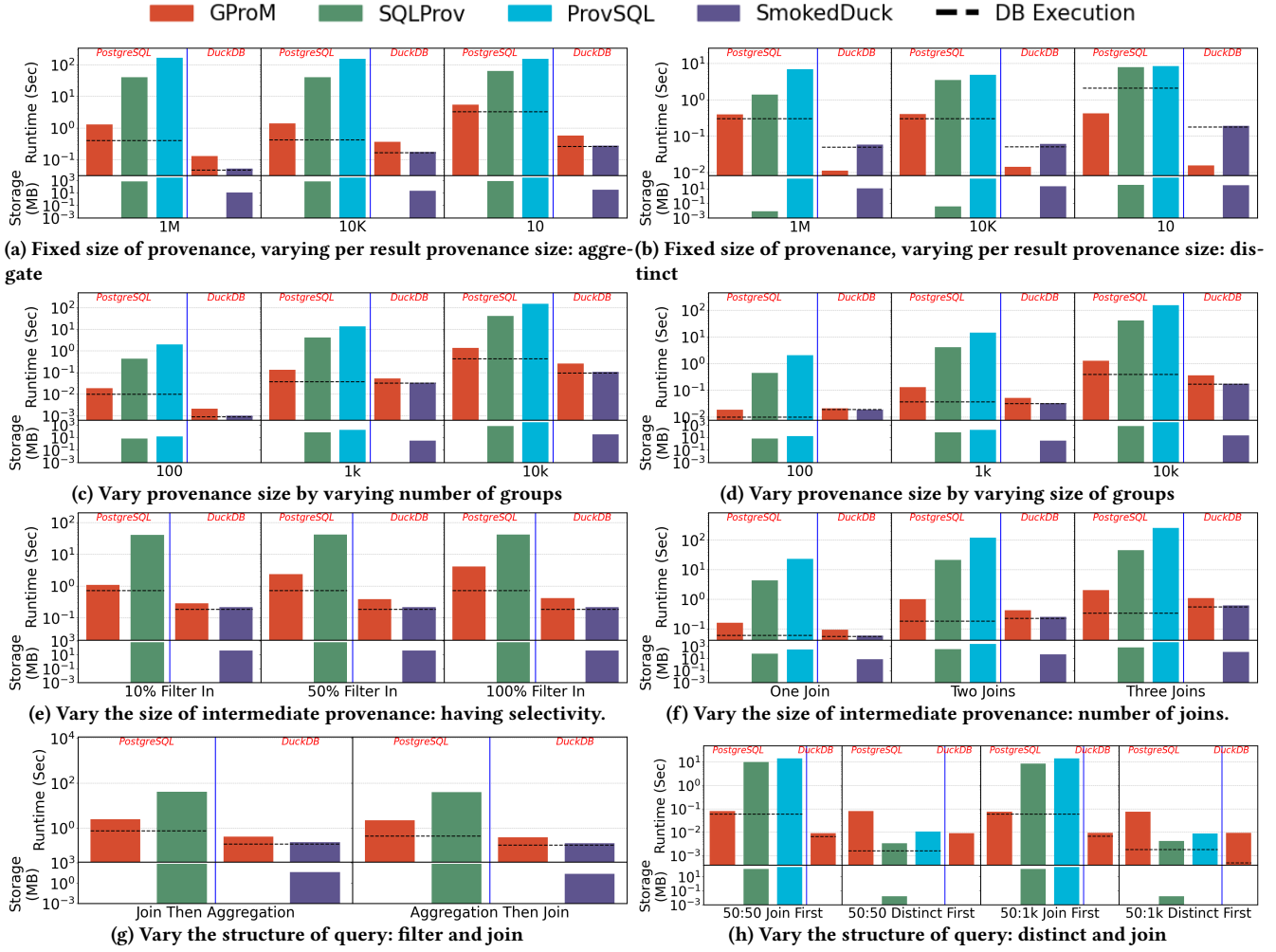


Figure 12: Use-case Specific Tasks - additional experiments

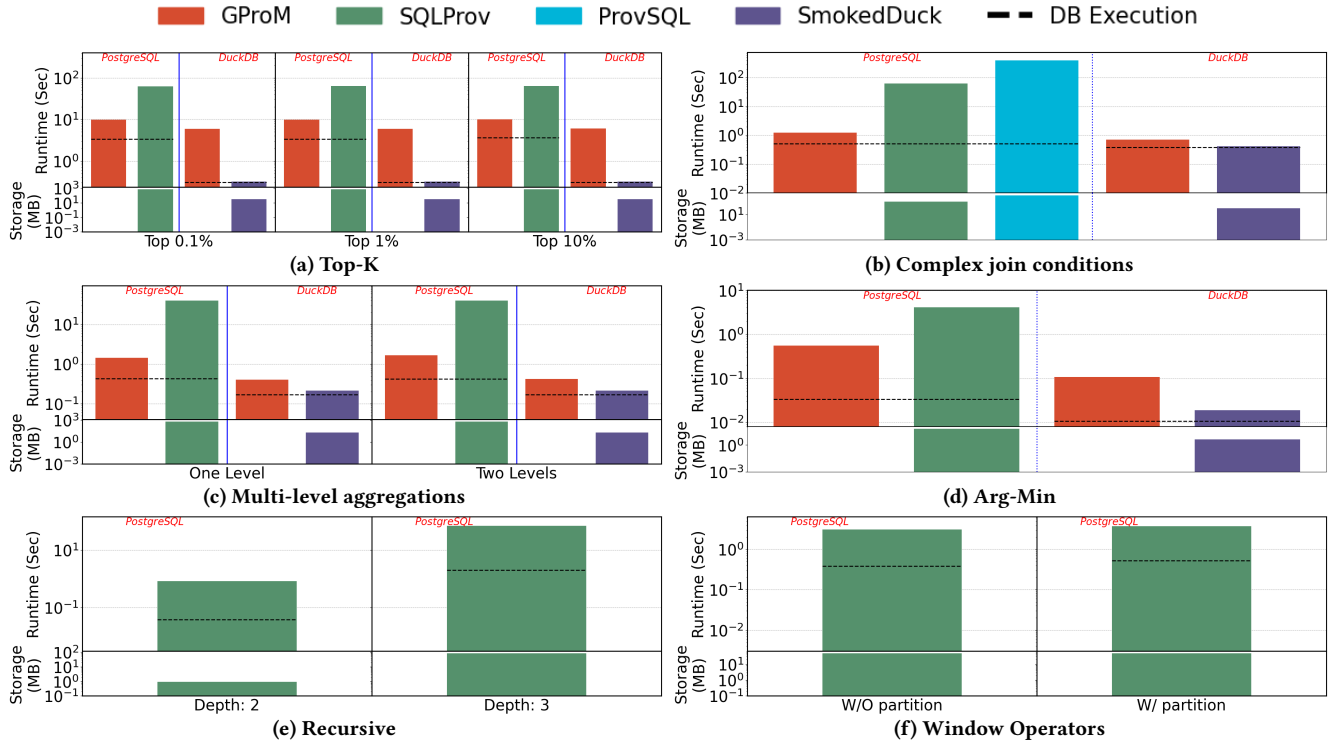


Figure 13: Query constructs & structure: additional experiments